

UNIVERSITÀ
DEGLI STUDI
DI PADOVA



UNIVERSITÀ DEGLI STUDI DI PADOVA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Sviluppo firmware per la gestione del sistema di riscaldamento di un prototipo di incubatrice neonatale

Relatore

Prof. Rampazzo Mirco

Laureando

Simonetto Giacomo
2109923

ANNO ACCADEMICO 2025-2026

Data di laurea 22/09/2026

Alla mia famiglia e ai miei amici.

Abstract

Il seguente elaborato finale descrive lo sviluppo del firmware per la gestione del sistema di riscaldamento e delle varie periferiche del prototipo di incubatrice neonatale costruito da Vanni Bincoletto nella sua tesi magistrale in bioingegneria [1].

Si inizia trattando la progettazione e realizzazione su breadboard del circuito necessario per collegare le varie periferiche del prototipo alla scheda programmabile Arduino Mega 2560. Si prosegue analizzando il firmware sviluppato per il controllo delle periferiche, il log dei dati su computer e il sistema di gestione della temperatura tramite un semplice controllore ad isteresi e un più evoluto controllore PID. Si conclude con la taratura dei parametri del controllore PID e l'analisi dei risultati ottenuti, evidenziando i vantaggi e gli svantaggi dei due controllori implementati.

Indice

Elenco delle figure	ix
Elenco delle tabelle	xi
Elenco dei codici	xiii
1 Introduzione	1
1.1 Contesto	1
1.2 Obiettivi della tesi	1
1.3 Struttura dell'elaborato	1
2 Sviluppo dell'hardware dell'incubatrice neonatale	3
2.1 Architettura del sistema	3
2.1.1 Descrizione della struttura fisica	3
2.1.2 Attuatori e relè	4
2.1.3 Pannello frontale e sensore SHT20	5
2.1.4 Scheda Arduino Mega 2560	6
2.2 Collegamento delle periferiche al microcontrollore	6
2.2.1 Connessioni dei relè ai diversi attuatori	6
2.2.2 Connessioni del pannello frontale e del sensore SHT20	6
2.2.3 Connessioni dei vari componenti al microcontrollore	7
3 Sviluppo del firmware per la gestione dell'incubatrice neonatale	11
3.1 Requisiti del firmware	11
3.2 Configurazione generale	11
3.3 Gestione degli attuatori	11
3.4 Gestione del pannello frontale	11
3.5 Gestione della comunicazione seriale al PC	12
3.6 Gestione del sensore di temperatura e umidità	12

4	Sviluppo del controllore per l'incubatrice neonatale	13
4.1	Controllore ad isteresi	13
4.2	Controllore PID	13
4.3	Confronto tra controllore ad isteresi e PID	13
A	Metodo dei minimi quadrati per il calcolo stabile della derivata	15
B	Codice sorgente del firmware	21
B.1	Files di configurazione del firmware	21
B.2	Files del modulo con le funzioni di utilità	25
B.3	Files del modulo per la gestione degli errori	27
B.4	Files del modulo per il controllo attuatori	31
B.5	Files del modulo per la gestione del display lcd	34
B.6	Files del modulo per la gestione del sensore di temperatura e umidità SHT20 . .	41
B.7	Files del modulo per la gestione delle comunicazioni seriali	46
B.8	Files del modulo per la gestione del controllore PID	50
B.9	File principale del firmware	55
	Bibliografia	65

Elenco delle figure

2.1	Foto degli attuatori	5
2.2	Foto dei relè e dello step down	5
2.3	Foto del pannello frontale	5
2.4	Foto del sensore SHT20	6
2.5	Foto della scheda Arduino Mega 2560	6
2.6	Schema circuitale dei relè e degli attuatori	7
2.7	Foto delle connessioni tra relè e attuatori	7
2.8	Schema circuitale del pannello frontale e del sensore SHT20	8
2.9	Foto delle connessioni nel pannello frontale	8
2.10	Schema circuitale del microcontrollore	9
2.11	Foto delle connessioni del microcontrollore	9

Elenco delle tabelle

2.1	Valori e misurazioni della corrente nei led di stato	9
-----	--	---

Elenco dei codici

1	File header con i parametri di configurazione	21
2	File sorgente con i parametri di configurazione	24
3	File header con le funzioni di utilità	25
4	File sorgente con le funzioni di utilità	26
5	File header con le funzioni per la gestione degli errori	27
6	File sorgente con le funzioni per la gestione degli errori	29
7	File header con le funzioni per il controllo degli attuatori	31
8	File sorgente con le funzioni per il controllo degli attuatori	32
9	File header con le funzioni per la gestione del display lcd	34
10	File sorgente con le funzioni per la gestione del display lcd	35
11	File header con le funzioni per la gestione del sensore SHT20	41
12	File sorgente con le funzioni per la gestione del sensore SHT20	42
13	File header con le funzioni per la gestione delle comunicazioni seriali	46
14	File sorgente con le funzioni per la gestione delle comunicazioni seriali	47
15	File header con le funzioni per la gestione del controllore PID	50
16	File sorgente con le funzioni per la gestione del controllore PID	51
17	File principale del firmware	55

Capitolo 1

Introduzione

1.1 Contesto

- scopo di una incubatrice neonatale

1.2 Obiettivi della tesi

- sviluppo trasversale sotto più ambiti (hardware, software, controllore)
- mettere in pratica le conoscenze teoriche viste a lezione
- verificare l'efficacia dei controllori pid per il controllo della temperatura

1.3 Struttura dell'elaborato

- analisi della parte hardware
- analisi della parte software
- analisi della parte di controllo

Capitolo 2

Sviluppo dell'hardware dell'incubatrice neonatale

2.1 Architettura del sistema

In questo capitolo vengono descritte le caratteristiche fisiche del prototipo di incubatrice neonatale. Si parte dalla descrizione della struttura fisica, si prosegue con la descrizione dei componenti elettronici presenti ed infine si analizzano le connessioni tra i vari componenti e il microcontrollore. Per una migliore organizzazione dell'elaborato, l'insieme delle componenti elettriche ed elettroniche è stato suddiviso in tre macrocategorie, affrontate nei successivi paragrafi:

1. attuatori e relè;
2. pannello frontale e sensore di temperatura e umidità SHT20;
3. scheda Arduino Mega 2560 con microcontrollore ATmega2560.

2.1.1 Descrizione della struttura fisica

L'incubatrice neonatale è composta da una camera interna principale ed un cassetto inferiore contenente i componenti elettronici e la vasca con l'acqua per l'umidificazione.

La camera interna è costituita con pareti laterali in polistirene espanso sinterizzato (ESP) sostenute ai quattro angoli da un telaio in legno. La base della camera interna è costituita da un pannello in legno ed è stato usato un pannello in policarbonato trasparente come rivestimento superiore. Il pannello superiore è incernierato alla struttura in legno lungo il lato lungo e permette il rapido accesso alla camera interna. Sul lato frontale sono presenti due sportelli che permettono un accesso rapido alla camera interna senza provocare grandi dispersioni di calore e umidità.

Il cassetto inferiore è realizzato con una struttura in legno estraibile dove sono posizionati i vari componenti elettronici e la vasca con l'acqua per l'umidificazione.

2.1.2 Attuatori e relè

Di seguito sono analizzati i vari attuatori presenti nel prototipo di incubatrice neonatale che permettono di agire sul sistema per modificarne le condizioni ambientali interne.

Riscaldatore con ventola associata

Per riscaldare l'aria della camera interna, è utilizzato un riscaldatore a resistenza elettrica da 100W. Esso è situato su un angolo della camera interna e sorretto da un supporto in alluminio attaccato al telaio in legno. Dietro al riscaldatore è presente una ventola che permette di spingere l'aria attraverso le resistenze elettriche del riscaldatore.

Umidificatore con ventola associata

Per l'umidificazione dell'aria della camera interna, si utilizza un umidificatore a ultrasuoni per generare vapore acqueo. Si trova nel cassetto inferiore all'interno di un contenitore in plastica ermetico che funge da serbatoio dell'acqua. Il vapore acqueo prodotto viene spinto dalla ventola all'interno di un tubo in plastica che collega il contenitore con la camera interna.

Ventola di omogeneizzazione dell'aria

Per favorire il ricircolo dell'aria nella camera interna, ed evitare la formazione di stratificazioni di temperatura e umidità, è presente una ventola di omogeneizzazione dell'aria. Si trova sull'angolo adiacente al riscaldatore ed è anch'essa sorretta da un supporto in alluminio.

Barra led

Per l'illuminazione della camera interna, è presente una barra led posta sul pannello trasparente superiore della camera interna.

Relè e step down per il controllo degli attuatori

I vari attuatori con le relative ventole sopra elencati richiedono tutti una tensione di alimentazione di 12 V. L'unico che fa eccezione è l'umidificatore che lavora a 24 V. Date le elevate correnti richieste, per garantire un funzionamento stabile e sicuro i vari gli attuatori a 12 V sono collegati ad un alimentatore da banco esterno opportunamente dimensionato, mentre l'umidificatore è alimentato dal suo alimentatore esterno da 24V.

Per poter essere gestiti dal microcontrollore, tra l'alimentatore e i vari attuatori, sono stati inseriti dei relè controllati con segnali a 5 V dalla scheda Arduino Mega. Siccome le bobine dei relè richiedono una corrente maggiore di quella che può essere erogata dal microcontrollore, è stato aggiunto un convertitore step down da 12 V a 5 V per alimentare le bobine dei relè a 5 V utilizzando l'energia erogata dall'alimentatore esterno a 12 V.

- | | |
|---|---|
| (a) Riscaldatore a resistenza elettrica con ventola posta dietro di esso. | (b) Umidificatore a ultrasuoni con ventola per spingere il vapore acqueo verso la camera interna. |
| (c) Ventola di omogeneizzazione dell'aria per favorire il ricircolo dell'aria nella camera interna. | (d) Barra LED per l'illuminazione della camera interna. |

Figura 2.1: Foto degli attuatori presenti nel prototipo di incubatrice neonatale.

Figura 2.2: Foto dei relè per il controllo degli attuatori del prototipo di incubatrice neonatale e dello step down per alimentare le bobine dei relè a 5 V.

2.1.3 Pannello frontale e sensore SHT20

Il pannello frontale del prototipo di incubatrice neonatale fornisce l'interfaccia utente con cui l'utilizzatore può interagire con il sistema. Si trova nella parte frontale del dispositivo ed è facilmente accessibile. Esso è composto dai seguenti componenti:

- un display LCD 20x4 con interfaccia I2C;
- 4 led di stato per indicare l'accensione del riscaldatore, l'accensione dell'umidificatore, la presenza di un allarme e la necessità di eseguire un refill del serbatoio dell'acqua dell'umidificatore;
- 3 switch a due posizioni per il controllo manuale del riscaldatore, dell'umidificatore e della barra led presente all'interno del prototipo di incubatrice neonatale;
- 2 potenziometri rotativi per la regolazione del setpoint di temperatura ed umidità.

Figura 2.3: Foto del pannello frontale del prototipo di incubatrice neonatale con i vari componenti presenti.

Il sensore di temperatura e umidità SHT20 è un sensore digitale che permette di misurare la temperatura e l'umidità relativa dell'ambiente in cui si trova. Esso è posizionato all'interno della camera interna del prototipo di incubatrice neonatale. Per i test effettuati, il sensore è stato posizionato a 20 cm dalle pareti laterali lunghe, a 17 cm dalla parete corta opposta al riscaldatore e alla ventola di omogeneizzazione dell'aria e a 10 cm di altezza dal piano della base della camera interna.

Figura 2.4: Sensore di temperatura e umidità SHT20 posizionato all'interno della camera interna del prototipo di incubatrice neonatale.

2.1.4 Scheda Arduino Mega 2560

Per gestire i vari componenti elettronici del prototipo di incubatrice neonatale, è stata utilizzata una scheda Arduino Mega 2560 con microcontrollore ATmega2560. Essa è stata scelta per la sua semplicità d'uso, la disponibilità di numerosi pin di input/output e l'elevato spazio di memoria disponibile per il firmware. La scheda è posizionata all'interno del cassetto inferiore del prototipo di incubatrice neonatale, insieme a tutti i cablaggi provenienti dai vari componenti elettronici. La scheda è alimentata via USB da un computer esterno su cui è possibile visualizzare i dati provenienti dal microcontrollore.

Figura 2.5: Scheda Arduino Mega 2560 posizionata all'interno del cassetto inferiore del prototipo di incubatrice neonatale.

2.2 Collegamento delle periferiche al microcontrollore

2.2.1 Connessioni dei relè ai diversi attuatori

Di seguito è riportato lo schema elettrico con le connessioni tra i relè e i vari attuatori presenti nel prototipo di incubatrice neonatale realizzato con il software Fritzing. Sono indicati anche i cavi di collegamento tra i relè e la scheda Arduino, insieme al convertitore step down per alimentare le bobine dei relè a 5 V.

Si evidenzia come l'umidificatore disponga di un suo sistema di alimentazione separato a 24 V, evidenziato dalla presenza del tratteggio sui cavi rosso e nero. Inoltre si nota che la massa dello step down e quella dei relè non sono immediatamente collegate tra di loro perché per distribuire la massa si utilizza la topologia a stella. Questo significa che tutti i componenti elettronici sono collegati a un unico punto di massa (mostrato nello schema dell'Arduino), eliminando i problemi di ground lifting e riducendo al minimo le interferenze dovute a sbalzi di corrente in corrispondenza delle commutazioni dei relè.

2.2.2 Connessioni del pannello frontale e del sensore SHT20

I componenti del pannello frontale e il sensore SHT20 sono collegati al microcontrollore tramite due cavi multipolari da 10 conduttori ciascuno. Il cablaggio è alloggiato in una canalina dedicata che unisce il cassetto inferiore al pannello frontale e alla camera interna dell'incubatrice.

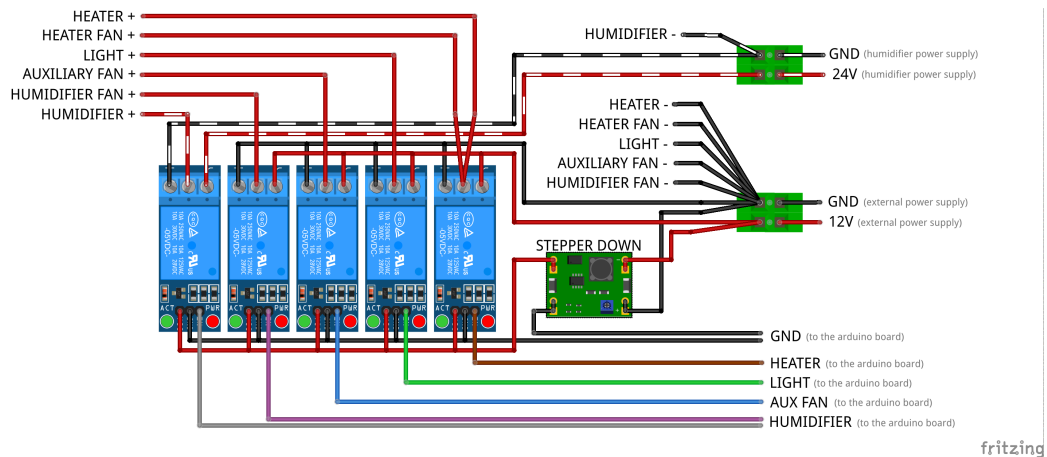


Figura 2.6: Schema elettrico con le connessioni dei relè e degli attuatori presenti nel prototipo di incubatrice neonatale.

Figura 2.7: Foto delle connessioni tra i relè e gli attuatori presenti nel prototipo di incubatrice neonatale.

Al fine di garantire affidabilità e agevolare le future operazioni di manutenzione, le connessioni verso il pannello frontale sono state modularizzate utilizzando dei connettori JST disponibili in laboratorio. In particolare sono stati scelti tre connettori a 4 pin per i segnali di controllo e un connettore a 2 pin per l'alimentazione. Le connessioni interne tra i componenti del pannello frontale, la catena di alimentazione e i rispettivi connettori sono state saldate con stagno per garantire un collegamento stabile e duraturo nel tempo.

Il sensore SHT20, per mantenere una certa flessibilità nella sua posizione all'interno della camera interna, è stato collegato ai cavi della canalina tramite una serie di 4 morsetti mam-muth. In questo modo è possibile scollegare il sensore, allungare o accorciare i cavi per poter posizionare agevolmente il sensore in diverse posizioni.

Per maggiore leggibilità, è stato scelto di tratteggiare i conduttori del cavo multipolare che collega il sensore SHT20 e il led di refill al microcontrollore. Si nota che non tutti i conduttori di questo cavo sono stati utilizzati.

2.2.3 Connessioni dei vari componenti al microcontrollore

Nel seguente schema elettrico sono riportate tutte le connessioni tra i cavi multipolari provenienti dai vari componenti del prototipo e i cavi di controllo dei relè con i pin del microcontrollore. Si nota la presenza del nodo principale di massa a cui convergono tutti i cavi di massa dei vari componenti.

Per limitare la corrente nei led di stato del pannello frontale, sono state inserite delle resistenze di bias. I relativi valori sono stati calcolati in modo da garantire una corrente di circa 10

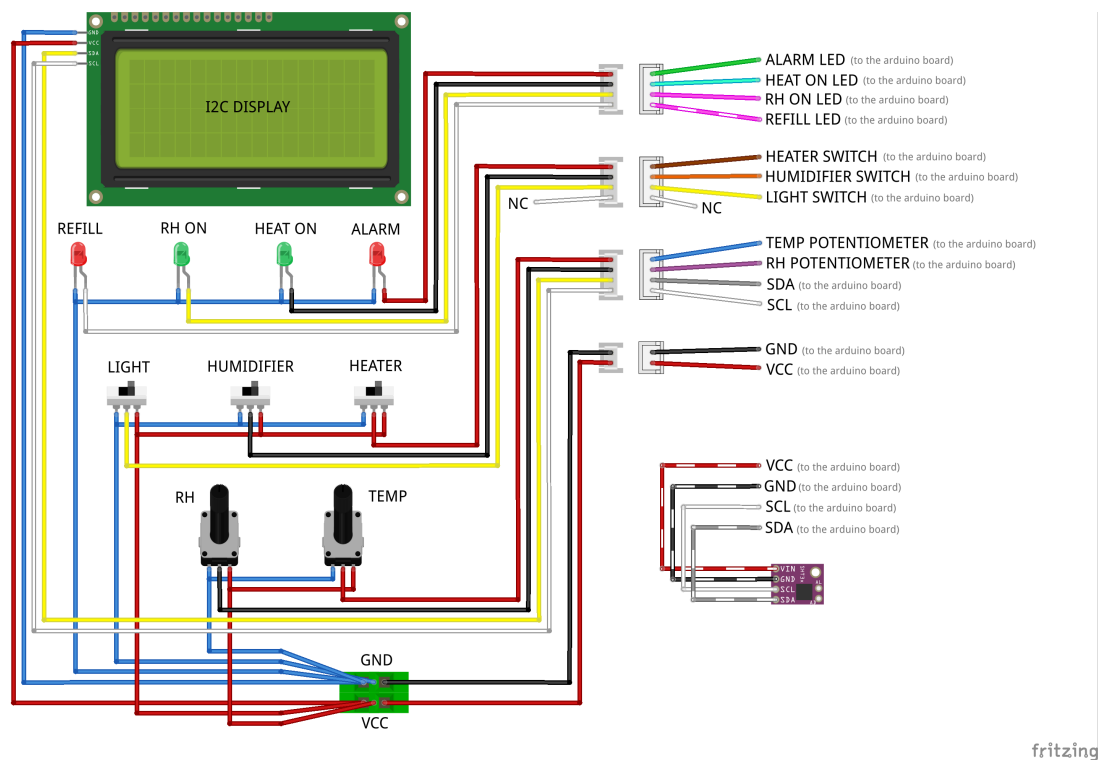


Figura 2.8: Schema elettrico con le connessioni del sensore di temperatura e umidità SHT20 e dei vari componenti presenti nel pannello frontale del prototipo di incubatrice neonatale.

Figura 2.9: Foto delle connessioni tra i vari componenti presenti nel pannello frontale del prototipo di incubatrice neonatale.

mA e sono poi state aggiustate in maniera empirica per ottenere una luminosità dei led adeguata. Di seguito sono riportati i valori e le misurazioni effettive della corrente nei led di stato. Si nota come le resistenze dei led di colore rosso siano più alte rispetto a quelle dei led di colore verde, in quanto, a parità di corrente, i led rossi risultano più luminosi rispetto a quelli verdi.

Led	Resistenza	Corrente
Allarme (rosso)	470 Ω	6.10 mA
Riscaldatore (verde)	220 Ω	12.2 mA
Umidificatore (verde)	220 Ω	12.2 mA
Refill (rosso)	470 Ω	6.11 mA

Tabella 2.1: Valori e misurazioni della corrente nei led di stato

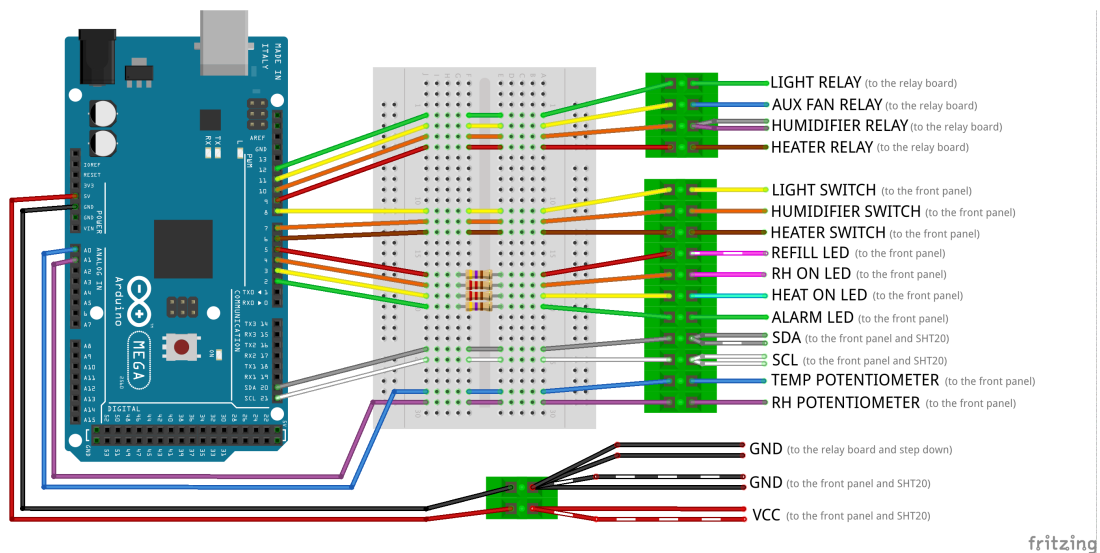


Figura 2.10: Schema elettrico con le connessioni dei cavi provenienti dai vari componenti al microcontrollore Arduino Mega 2560.

Figura 2.11: Foto delle connessioni dei cavi provenienti dai vari componenti al microcontrollore Arduino Mega 2560.

Capitolo 3

Sviluppo del firmware per la gestione dell'incubatrice neonatale

3.1 Requisiti del firmware

3.2 Configurazione generale

- variabili globali di stato
- files e moduli del firmware
- gestione tipo di controllo (manuale o automatico)

3.3 Gestione degli attuatori

- controllo manuale o automatico degli attuatori
- gestione centralizzata degli attuatori per aggiornare le variabili di stato
- gestione della ventola di omogeneizzazione della temperatura
- gestione del refill dell'acqua per il sistema di umidificazione

3.4 Gestione del pannello frontale

- lettura degli switch, con impostazione controllo manuale
- lettura dei potenziometri, con filtri per ridurre il rumore
- gestione del display lcd con temporizzazioni
- gestione degli errori e gestione del led di allarme

3.5 Gestione della comunicazione seriale al PC

- comunicazioni seriali
- codice matlab

3.6 Gestione del sensore di temperatura e umidità

- gestione non bloccante
- utilizzo di una macchina a stati

Capitolo 4

Sviluppo del controllore per l'incubatrice neonatale

4.1 Controllore ad isteresi

- definizione del controllore ad isteresi
- implementazione di aggiornamento immediato
- test di funzionamento con grafico e prestazioni

4.2 Controllore PID

- definizione del controllore PID
- implementazione dell'aggiornamento dello stato
- gestione del ciclo pwm e salvataggio delle misurazioni
- test di funzionamento con grafico e prestazioni

4.3 Confronto tra controllore ad isteresi e PID

Appendice A

Metodo dei minimi quadrati per il calcolo stabile della derivata

Di seguito viene illustrato il procedimento matematico effettuato per ottenere la formula utilizzata nel controllore PID per il calcolo del contributo derivativo.

Il calcolo della derivata di una grandezza fisica misurata da un sensore risulta particolarmente delicato in quanto è fortemente soggetto al rumore delle misurazioni. Il semplice calcolo del rapporto incrementale tra due campioni consecutivi può portare a rapide variazioni della derivata che potrebbero indurre problemi di stabilità del controllore. Per mitigare le fluttuazioni istantanee si è scelto di utilizzare il metodo dei minimi quadrati ed effettuare una regressione lineare su un set più ampio di dati in modo da compensare le fluttuazioni istantanee e ottenere una stima più stabile della derivata.

Notazioni e convenzioni utilizzate

Nei calcoli successivi si adottano le seguenti notazioni:

- y_i : misurazione del sensore all'istante di tempo x_i
- $a_0 + a_1 x_i$: retta di regressione lineare, dove a_0 è intercetta e a_1 è il coefficiente angolare della retta di regressione lineare, che rappresenta la derivata da calcolare

Per calcolare la derivata ad un certo istante di tempo x_i , si considerano solo gli ultimi N campioni misurati dal sensore. Scelto x_k l'istante di tempo associato all'ultima misurazione y_j , l'indice i dei campioni considerati varia in $[j - (N - 1), j]$ con $j \in \mathbb{N}$. Siccome gli indici delle sommatorie sono variabili mute, nei successivi passaggi si considera senza perdita di generalità $j = N - 1$ in modo che i varia da 0 a $N - 1$.

Procedimento matematico

L'errore e_i tra la misurazione y_i e la retta di regressione lineare $a_0 + a_1 x_i$ viene definito come:

$$e_i = y_i - (a_0 + a_1 x_i) \quad (\text{A.1})$$

Si definisce quindi l'errore quadratico totale $S(a_0, a_1)$ come segue:

$$S(a_0, a_1) = \sum_{i=0}^{N-1} e_i^2 = \sum_{i=0}^{N-1} (y_i - (a_0 + a_1 x_i))^2 \quad (\text{A.2})$$

I parametri a_0 e a_1 della retta di regressione lineare che minimizzano l'errore quadratico totale $S(a_0, a_1)$ si ottengono ponendo a zero le derivate parziali di $S(a_0, a_1)$ rispetto a a_0 e a_1 e risolvendo il sistema lineare di due equazioni a due incognite a_0 e a_1 così ottenuto.

$$\frac{\partial S(a_0, a_1)}{\partial a_0} = -2 \sum_{i=0}^{N-1} (y_i - (a_0 + a_1 x_i)) = \quad (\text{A.3})$$

$$= -2 \left(\sum_{i=0}^{N-1} y_i - \sum_{i=0}^{N-1} a_0 - \sum_{i=0}^{N-1} a_1 x_i \right) = \quad (\text{A.4})$$

$$= -2 \left(\sum_{i=0}^{N-1} y_i - a_0 \sum_{i=0}^{N-1} 1 - a_1 \sum_{i=0}^{N-1} x_i \right) = \quad (\text{A.5})$$

$$= -2 (N\bar{y} - a_0 N - a_1 N\bar{x}) = \quad (\text{A.6})$$

$$= -2N (\bar{y} - a_0 - a_1 \bar{x}) \quad (\text{A.7})$$

$$\frac{\partial S(a_0, a_1)}{\partial a_1} = -2 \sum_{i=0}^{N-1} x_i (y_i - (a_0 + a_1 x_i)) = \quad (\text{A.8})$$

$$= -2 \left(\sum_{i=0}^{N-1} x_i y_i - \sum_{i=0}^{N-1} a_0 x_i - \sum_{i=0}^{N-1} a_1 x_i^2 \right) = \quad (\text{A.9})$$

$$= -2 \left(\sum_{i=0}^{N-1} x_i y_i - a_0 \sum_{i=0}^{N-1} x_i - a_1 \sum_{i=0}^{N-1} x_i^2 \right) = \quad (\text{A.10})$$

$$= -2 \left(\sum_{i=0}^{N-1} x_i y_i - a_0 N\bar{x} - a_1 \sum_{i=0}^{N-1} x_i^2 \right) \quad (\text{A.11})$$

Si procede quindi a risolvere il sistema di equazioni lineari così ottenuto per sostituzione:

$$\begin{cases} \frac{\partial S(a_0, a_1)}{\partial a_0} = 0 \\ \frac{\partial S(a_0, a_1)}{\partial a_1} = 0 \end{cases} \Rightarrow \begin{cases} 2N (\bar{y} - a_0 - a_1 \bar{x}) = 0 \\ -2 \left(\sum_{i=0}^{N-1} x_i y_i - a_0 N \bar{x} - a_1 \sum_{i=0}^{N-1} x_i^2 \right) = 0 \end{cases} \quad (\text{A.12})$$

$$-2N (\bar{y} - a_0 - a_1 \bar{x}) = 0 \Rightarrow a_0 = \bar{y} - a_1 \bar{x} \quad (\text{A.13})$$

$$-2 \left(\sum_{i=0}^{N-1} x_i y_i - a_0 N \bar{x} - a_1 \sum_{i=0}^{N-1} x_i^2 \right) = 0 \quad (\text{A.14})$$

$$\sum_{i=0}^{N-1} x_i y_i - (\bar{y} - a_1 \bar{x}) N \bar{x} - a_1 \sum_{i=0}^{N-1} x_i^2 = 0 \quad (\text{A.15})$$

$$\sum_{i=0}^{N-1} x_i y_i - N \bar{y} \bar{x} + a_1 N \bar{x}^2 - a_1 \sum_{i=0}^{N-1} x_i^2 = 0 \quad (\text{A.16})$$

$$\sum_{i=0}^{N-1} x_i y_i - N \bar{y} \bar{x} - a_1 \left(\sum_{i=0}^{N-1} x_i^2 - N \bar{x}^2 \right) = 0 \quad (\text{A.17})$$

$$(\text{A.18})$$

$$a_1 = \frac{\sum_{i=0}^{N-1} x_i y_i - N \bar{y} \bar{x}}{\sum_{i=0}^{N-1} x_i^2 - N \bar{x}^2} \quad (\text{A.19})$$

Si assume che gli istanti di tempo x_i sono tutti equidistanti, ovvero che formano una sequenza continua dei multipli del tempo di campionamento T_s a partire dall'istante iniziale $k T_s$ dove $k = j - (N - 1)$. I campioni x_i possono essere quindi espressi in funzione dei due indici i e k :

$$x_i = (j - (N - 1)) T_s + i T_s = (k + i) T_s \quad \text{con } \bar{x} = \frac{1}{N} \sum_{i=0}^{N-1} x_i = \frac{T_s}{N} \sum_{i=0}^{N-1} (k + i) \quad (\text{A.20})$$

È possibile quindi riscrivere la formula (A.19) con la sostituzione (A.20) individuata sopra:

$$a_1 = \frac{T_s \sum_{i=0}^{N-1} (k + i) y_i - N \bar{y} \left(\frac{T_s}{N} \sum_{i=0}^{N-1} (k + i) \right)}{T_s^2 \sum_{i=0}^{N-1} (k + i)^2 - N \left(\frac{T_s}{N} \sum_{i=0}^{N-1} (k + i) \right)^2} \quad (\text{A.21})$$

Analizzando separatamente il numeratore e il denominatore della formula (A.19) si ottengono le seguenti semplificazioni fino ad ottenere la formula finale utilizzata nell'implementazione del controllore PID:

$$Num. = T_s \sum_{i=0}^{N-1} (k+i) y_i - N\bar{y} \left(\frac{T_s}{N} \sum_{i=0}^{N-1} (k+i) \right) = \quad (A.22)$$

$$= T_s \left(k \sum_{i=0}^{N-1} y_i + \sum_{i=0}^{N-1} i \cdot y_i \right) - N\bar{y} \cdot \frac{T_s}{N} \cdot \left(\sum_{i=0}^{N-1} i + \sum_{i=0}^{N-1} k \right) = \quad (A.23)$$

$$= T_s \left(kN\bar{y} + \sum_{i=0}^{N-1} i \cdot y_i - \bar{y} \left(\frac{N(N-1)}{2} + kN \right) \right) = \quad (A.24)$$

$$= T_s \left(kN\bar{y} + \sum_{i=0}^{N-1} i \cdot y_i - \frac{N(N-1)}{2} \cdot \bar{y} - kN\bar{y} \right) = \quad (A.25)$$

$$= T_s \left(\sum_{i=0}^{N-1} i \cdot y_i - \frac{N(N-1)}{2} \cdot \bar{y} \right) = \quad (A.26)$$

$$= T_s \left(\sum_{i=0}^{N-1} i \cdot y_i - \frac{N-1}{2} \cdot \sum_{i=0}^{N-1} y_i \right) \quad (A.27)$$

$$Den. = T_s^2 \sum_{i=0}^{N-1} (k+i)^2 - N \left(\frac{T_s}{N} \sum_{i=0}^{N-1} (k+i) \right)^2 = \quad (A.28)$$

$$= T_s^2 \left(\sum_{i=0}^{N-1} (k^2 + 2ik + i^2) - \frac{T_s^2}{N} \left(\sum_{i=0}^{N-1} (k+i) \right)^2 \right) = \quad (A.29)$$

$$= T_s^2 \left(k^2N + 2k \sum_{i=0}^{N-1} i + \sum_{i=0}^{N-1} i^2 - \frac{1}{N} \left(kN + \sum_{i=0}^{N-1} i \right)^2 \right) = \quad (A.30)$$

$$= T_s^2 \left(k^2N + 2k \frac{N(N-1)}{2} + \frac{N(N-1)(2N-1)}{6} - \frac{1}{N} \left(kN + \frac{N(N-1)}{2} \right)^2 \right) = \quad (A.31)$$

$$= T_s^2 \left(k^2N + kN^2 - kN + \frac{2N^3 - 3N^2 + N}{6} + \right. \\ \left. - \frac{1}{N} \cdot \left(k^2N^2 + kN^3 - kN^2 + \frac{N^4 - 2N^3 + N^2}{4} \right) \right) = \quad (A.32)$$

$$= \frac{T_s^2}{12} \cdot (12k^2N + 12kN^2 - 12kN + 4N^3 - 6N^2 + 2N + \\ + 12k^2N - 12kN^2 + 12kN - 3N^3 + 6N^2 - 3N) = \quad (A.33)$$

$$= \frac{T_s^2}{12} (N^3 - N) = \frac{T_s^2}{12} \cdot N(N-1)(N+1) \quad (A.34)$$

Unendo i due termini (A.27) e (A.34) così ottenuti, si arriva alla formula finale utilizzata nell'implementazione del controllore PID.

$$a_1 = \frac{T_s \sum_{i=0}^{N-1} (k+i) y_i - N\bar{y} \left(\frac{T_s}{N} \sum_{i=0}^{N-1} (k+i) \right)}{T_s^2 \sum_{i=0}^{N-1} (k+i)^2 - N \left(\frac{T_s}{N} \sum_{i=0}^{N-1} (k+i) \right)^2} = \quad (\text{A.35})$$

$$= \frac{T_s \left(\sum_{i=0}^{N-1} i \cdot y_i - \frac{N-1}{2} \cdot \sum_{i=0}^{N-1} y_i \right)}{\frac{T_s^2}{12} \cdot N(N-1)(N+1)} = \quad (\text{A.36})$$

$$= \frac{12}{T_s \cdot N(N-1)(N+1)} \cdot \left(\sum_{i=0}^{N-1} i \cdot y_i - \frac{N-1}{2} \cdot \sum_{i=0}^{N-1} y_i \right) = \quad (\text{A.37})$$

$$= \frac{12}{T_s \cdot N(N^2-1)} \cdot \sum_{i=0}^{N-1} i \cdot y_i - \frac{6}{T_s \cdot N(N+1)} \cdot \sum_{i=0}^{N-1} y_i \quad (\text{A.38})$$

Appendice B

Codice sorgente del firmware

B.1 Files di configurazione del firmware

Codice 1: File header con i parametri di configurazione

```
1  /**
2   * @file config.h
3   * @author Giacomo Simonetto
4   * @brief Configurazione di parametri e variabili globali di stato.
5   *
6   * Dichiarazione dei parametri di configurazione e delle variabili globali
7   * di stato dell'incubatrice neonatale.
8   */
9
10 #ifndef CONFIG_H
11 #define CONFIG_H
12
13 #include <Arduino.h>
14 #include <Wire.h>
15 #include <LiquidCrystal_I2C.h>
16 #include "sht20.h"
17
18 // ----- configurazione pin dell'arduino mega -----
19 // -----
20
21 #define ALARM_LED 2 // led rosso di allarme
22 #define HEAT_LED 3 // led verde di stato riscaldatore
23 #define RH_LED 4 // led verde di stato umidificatore
24 #define REFILL_LED 5 // led rosso di refill acqua
25
26 #define HEAT_SW 6 // interruttore riscaldatore
27 #define RH_SW 7 // interruttore umidificatore
28 #define LIGHT_SW 8 // interruttore illuminazione
29
30 #define HEAT_RELAY 9 // relè riscaldatore
31 #define RH_RELAY 10 // relè umidificatore
32 #define FAN_RELAY 11 // relè ventola ausiliaria
33 #define LIGHT_RELAY 12 // relè illuminazione
34
35 #define TEMP_POT A0 // potenziometro temperatura
36 #define RH_POT A1 // potenziometro umidità
37
38
39 // ----- configurazione parametri programma -----
40 // -----
41
42 // parametri per potenziometro e setpoint di temperatura
43 #define TEMP_POT_THLD 5 // soglia per soppressione del rumore
44 #define TEMP_POT_MIN 10 // valore minimo del potenziometro (misurato 3-4)
45 #define TEMP_POT_MAX 1015 // valore massimo del potenziometro (misurato 1019-1020)
```

```

48 #define TEMP_DEF_SP 3600 // setpoint iniziale temperatura
49 #define TEMP_MIN_SP 2500 // valore minimo del setpoint
50 #define TEMP_MAX_SP 4000 // valore massimo del setpoint
51 #define TEMP_STEP_SP 10 // passo di variazione del setpoint
52 #define TEMP_ERR_THLD 200 // soglia di errore per temperatura
53 #define TEMP_ERR_HYST 20 // isteresi per errore di temperatura
54
55 // parametri potenziometro e setpoint di umidità
56 #define RH_POT_THLD 5 // soglia per soppressione del rumore
57 #define RH_POT_MIN 10 // valore minimo del potenziometro (misurato 3-4)
58 #define RH_POT_MAX 1015 // valore massimo del potenziometro (misurato 1019-1020)
59 #define RH_DEF_SP 2000 // setpoint iniziale umidità
60 #define RH_MIN_SP 2000 // valore minimo del setpoint
61 #define RH_MAX_SP 9000 // valore massimo del setpoint
62 #define RH_STEP_SP 100 // passo di variazione del setpoint
63 #define RH_ERR_THLD 1000 // soglia di errore per umidità
64 #define RH_ERR_HYST 100 // isteresi per errore di umidità
65
66 // parametri per la gestione dei timer
67 #define SW_READ_PERIOD 50 // tempo di campionamento degli switch
68 #define TEMP_POT_READ_PERIOD 100 // tempo di campionamento del temp_pot
69 #define RH_POT_READ_PERIOD 100 // tempo di campionamento dell'rh_pot
70 #define SP_UPDATE_DELAY 1300 // ritardo di aggiornamento del setpoint
71 #define SHT20_READ_PERIOD 1000 // tempo di campionamento dell'SHT20
72 #define LCD_UPDATE_PERIOD 1000 // tempo di aggiornamento dell'lcd
73 #define ERR_TIMER 1100 // tempo di durata degli errori temporizzati
74 #define REFILL_INTERVAL 3600000UL // durata dell'acqua prima del refill
75 // per ora è impostato genericamente a 1 ora, poi servirà andare a tararla
76 // utilizzando il dato di 0,1017 g/s di acqua nebulizzata dalla tesi di Vanni
77
78 // parametri per il controllo della temperatura
79 #define TEMP_CTRL 2 // 0 = no control | 1 = hysteresis | 2 = PID
80 #define TEMP_HYS_THLD 25 // soglia di isteresi per temperatura
81 #define TEMP_PID_KP 50.0f // guadagno proporzionale del PID per temperatura
82 #define TEMP_PID_KI 0.5f // guadagno integrale del PID per temperatura
83 #define TEMP_PID_KD 20.0f // guadagno derivativo del PID per temperatura
84 #define TEMP_PID_KW 0.0f // guadagno anti-windup del PID per temperatura
85 #define TEMP_PID_WINDUP 1 // 0 = no windup | 1 = clamping | 2 = back calculation
86
87 // parametri per il controllo dell'umidità
88 #define RH_CTRL 2 // 0 = no control | 1 = hysteresis | 2 = PID
89 #define RH_HYS_THLD 200 // soglia di isteresi per umidità
90 #define RH_PID_KP 0.0f // guadagno proporzionale del PID per umidità
91 #define RH_PID_KI 0.0f // guadagno integrale del PID per umidità
92 #define RH_PID_KD 0.0f // guadagno derivativo del PID per umidità
93 #define RH_PID_KW 0.0f // guadagno anti-windup del PID per umidità
94 #define RH_PID_WINDUP 0 // 0 = no windup | 1 = clamping | 2 = back calculation
95
96 // parametri per la gestione del segnale pwm
97 #define PWM_PERIOD 8000 // periodo del pwm per controllo attuatori (in ms) (max 30 sec)
98 #define PWM_MIN_TIME_ON 500 // intervallo minimo di accensione attuatori (in ms)
99 #define PWM_MIN_TIME_OFF 500 // intervallo minimo di spegnimento attuatori (in ms)
100
101 // parametri generali per i controllori pid
102 #define PID_MIN_OUTPUT 0 // limite minimo dell'output del PID
103 #define PID_MAX_OUTPUT 100 // limite massimo dell'output del PID
104 #define PID_DATA_PERIOD SHT20_READ_PERIOD // periodo di acquisizione dei dati
105 #define PID_UPDATE_PERIOD PWM_PERIOD // periodo di aggiornamento dell'output
106
107 // ----- configurazione flag del programma -----
108 // -----
109
110
111 // stati relay, switch e led
112 #define RELAY_OFF HIGH // relè spento (logica invertita)
113 #define RELAY_ON LOW // relè acceso (logica invertita)
114 #define LED_OFF LOW // led spento
115 #define LED_ON HIGH // led acceso
116
117 // modalità di controllo della temperatura e dell'umidità
118 #define AUTO_CTRL 0 // controllo automatico tramite PID o isteresi
119 #define MANUAL_CTRL 1 // controllo manuale tramite switch
120
121
122 // ----- sensori e periferiche dell'incubatrice -----
123 // -----
124
125
126 // sensore SHT20 per temperatura e umidità

```

```

127 extern sht20 sensor;
128
129 // display lcd 20x4 con interfaccia I2C
130 extern LiquidCrystal_I2C lcd;
131
132
133 // -----
134 // ----- definizione struct per variabili di stato dell'incubatrice -----
135 // -----
136
137 // variabili globali di stato
138 struct status {
139     // misurazioni e setpoint di temperatura e umidità
140     // memorizzati come interi a 4 cifre (es. 36.25°C = 3625, 20.00% = 2000)
141     int16_t temp_sht20; // temperatura rilevata dal sensore SHT20
142     int16_t rh_sht20; // umidità rilevata dal sensore SHT20
143     int16_t temp_setpoint; // setpoint temperatura
144     int16_t rh_setpoint; // setpoint umidità
145
146     // valori dei potenziometri
147     int16_t temp_pot_value; // valore potenziometro temperatura
148     int16_t rh_pot_value; // valore potenziometro umidità
149
150     // duty cycle per il controllo tramite pwm della temperatura e dell'umidità
151     uint16_t temp_pwm_value; // duty cycle per il pwm della temperatura (in ms)
152     uint16_t rh_pwm_value; // duty cycle per il pwm dell'umidità (in ms)
153
154     // codice di errore
155     uint16_t error_code;
156
157     // stato dei relè
158     bool heat_relay; // stato relay e led riscaldatore
159     bool rh_relay; // stato relay e led umidificatore
160     bool fan_relay; // stato relay ventola ausiliaria
161     bool light_relay; // stato relay illuminazione
162
163     // stato degli switch
164     bool heat_sw; // stato interruttore riscaldatore
165     bool rh_sw; // stato interruttore umidificatore
166     bool light_sw; // stato interruttore illuminazione
167
168     // stato dei led (esclusi led del riscaldatore e dell'umidificatore)
169     bool alarm_led; // stato led di allarme
170     bool refill_led; // stato led di refill acqua
171
172     // variabile per disabilitare il controllo automatico della temperatura
173     // e dell'umidità quando si agisce manualmente sugli switch
174     bool manual_ctrl;
175 };
176
177 // variabili per gestione dei timer
178 struct timers {
179     // timer per letture periferiche (switch, potenziometri, sensore SHT20)
180     uint32_t heat_sw; // lettura switch riscaldatore
181     uint32_t rh_sw; // lettura switch umidificatore
182     uint32_t light_sw; // lettura switch illuminazione
183     uint32_t temp_pot; // lettura potenziometro temperatura
184     uint32_t rh_pot; // lettura potenziometro umidità
185
186     // timer per aggiornamento setpoint e display
187     uint32_t temp_setpoint; // aggiornamento ritardato del setpoint temperatura
188     uint32_t rh_setpoint; // aggiornamento ritardato del setpoint umidità
189     uint32_t lcd_update; // aggiornamento display lcd
190
191     // timer di gestione del refill dell'acqua
192     uint32_t last_rh_on; // timestamp ultima accensione umidificatore
193     uint32_t refill_counter; // contatore accensione umidificatore
194
195     // timer per segnale pwm
196     uint32_t start_pwm; // timestamp inizio ciclo pwm
197 };
198
199 // parametri del controllore PID
200 struct pid {
201     // guadagni del controllore PID
202     float kp; // guadagno proporzionale
203     float ki; // guadagno integrale
204     float kd; // guadagno derivativo
205     float kw; // guadagno windup

```

```

206
207 // componenti del controllore PID
208 float proportional; // componente proporzionale
209 float integral; // componente integrale
210 float derivative; // componente derivativa
211 float output; // output puro del PID non limitato
212
213 // variabili e accumulatori intermedi per calcolo PID
214 float deriv_c1; // primo coefficiente per calcolo derivata
215 float deriv_c2; // secondo coefficiente per calcolo derivata
216 int32_t sum1; // somma degli errori tra setpoint e valore misurato
217 int32_t sum2; // somma dei valori misurati
218 int32_t sum3; // somma dei prodotti tra indice e valore misurato
219
220 // numero di misurazioni
221 uint8_t data_count; // misurazioni ricevute ed elaborate
222 uint8_t expected_data_count; // misurazioni attese
223
224 // metodo di anti-windup da utilizzare
225 uint8_t anti_windup; // 0 = no windup / 1 = clamping / 2 = back calculation
226 };
227
228 // -----
229 // ----- variabili di stato dell'incubatrice -----
230 // -----
231
232 // variabili globali di stato dell'incubatrice neonatale
233 extern struct status status; // variabili di stato dell'incubatrice
234 extern struct timers timers; // variabili per i timer dell'incubatrice
235 extern struct pid temp_pid; // variabili per il PID della temperatura
236 extern struct pid rh_pid; // variabili per il PID dell'umidità
237
238 #endif // CONFIG_H

```

Codice 2: File sorgente con i parametri di configurazione

```

1 /**
2  * @file config.cpp
3  * @author Giacomo Simonetto
4  * @brief Configurazione di parametri e variabili globali di stato.
5  *
6  * Dichiarazione dei parametri di configurazione e delle variabili globali
7  * di stato dell'incubatrice neonatale.
8  */
9
10 #include "config.h"
11
12 // variabili globali di stato dell'incubatrice neonatale
13 struct status status; // variabili di stato dell'incubatrice
14 struct timers timers; // variabili per i timer dell'incubatrice
15 struct pid temp_pid; // variabili per il PID della temperatura
16 struct pid rh_pid; // variabili per il PID dell'umidità
17
18 // sensore SHT20 per temperatura e umidità
19 sht20 sensor(SHT20_READ_PERIOD);
20
21 // display lcd 20x4 con interfaccia I2C
22 LiquidCrystal_I2C lcd(0x27, 20, 4);

```

B.2 Files del modulo con le funzioni di utilità

Codice 3: File header con le funzioni di utilità

```
1  /**
2   * @file utils.h
3   * @author Giacomo Simonetto
4   * @brief Funzioni di utilità per l'incubatrice neonatale
5   *
6   * Dichiarazione delle funzioni di utilità per l'incubatrice neonatale.
7   */
8
9  #ifndef UTILS_H
10 #define UTILS_H
11
12 #include "config.h"
13
14 // -----
15 // ----- gestione della conversione da intero a 4 cifre a stringa -----
16 // -----
17
18 // buffer per conversione da intero a 4 cifre a stringa
19 extern char float_buffer[6];
20
21 /**
22  * @brief Converte una misurazione in stringa formattata correttamente.
23  *
24  * Converte i valori di temperatura ed umidità nella loro rappresentazione
25  * in stringa con due cifre intere e due decimali per la stampa sul display
26  * lcd e sul monitor seriale.
27  *
28  * @param value il numero da convertire come intero a 4 cifre
29  * @return puntatore al float_buffer contenente la stringa formattata
30  */
31 char *data_to_string(int16_t value);
32
33
34 // -----
35 // ----- calcolo dell'umidità relativa al raggiungimento del setpoint -----
36 // -----
37
38 /**
39  * @brief Calcola l'umidità relativa risultante al raggiungimento del setpoint
40  * di temperatura.
41  *
42  * Il valore di umidità relativa è calcolato in funzione della temperatura,
43  * per cui variando la temperatura cambia anche il valore di umidità relativa.
44  * Per gestire correttamente l'accensione e lo spegnimento dell'umidificatore
45  * è necessario confrontare il setpoint di umidità impostato con l'umidità
46  * relativa che si avrebbe al raggiungimento del setpoint di temperatura.
47  * Per calcolare tale valore si utilizza la seguente formula:
48  *
49  * 
$$rh2 = rh1 * \exp(4283.78 * (t1 - t2) / (243.12 + t1) / (243.12 + t2))$$

50  *
51  * - t1: temperatura dello stato 1 in °C (attuale - misurata dallo SHT20)
52  * - t2: temperatura dello stato 2 in °C (desiderata - dal setpoint)
53  * - rh1: umidità relativa dello stato 1 (attuale - misurata dallo SHT20)
54  * - rh2: umidità relativa dello stato 2 (desiderata)
55  *
56  * fonte:
57  * ↪ https://sensirion.com/media/documents/A419127A/6836C0D2/Sensirion\_AppNotes\_Humidity\_Sensors\_at\_a\_Glance.pdf
58  *
59  * Per rendere il calcolo più efficiente, impiegando solo numeri interi nella
60  * rappresentazione a 4 cifre, la formula viene riscritta come segue:
61  *
62  * 
$$rh2 = rh1/100 * \exp(4283.78 * (t1/100 - t2/100) / (243.12 + t1/100) / (243.12 + t2/100)) * 100$$

63  * 
$$= rh1 * \exp(4283.78 * (t1 - t2)/100 / ((24312 + t1)/100) / ((24312 + t2)/100))$$

64  * 
$$= rh1 * \exp(4283.78 * (t1 - t2) / 100 / (24312 + t1) * 100 / (24312 + t2) * 100)$$

65  * 
$$= rh1 * \exp(4283.78 * (t1 - t2) / (24312 + t1) / (24312 + t2) * 100)$$

66  * 
$$= rh1 * \exp(428378 * (t1 - t2) / (24312 + t1) / (24312 + t2))$$

67  *
68  * @param rh1 umidità attuale misurata dal sensore SHT20
69  * @param t1 temperatura attuale misurata dal sensore SHT20
70  * @param t2 setpoint di temperatura desiderato
71  * @return int16_t umidità relativa al raggiungimento del setpoint di temperatura
72  */
73 int16_t rh_at_temp_setpoint(int16_t rh1, int16_t t1, int16_t t2);
```

```

73
74 #endif // UTILS_H

```

Codice 4: File sorgente con le funzioni di utilità

```

1  /**
2   * @file utils.cpp
3   * @author Giacomo Simonetto
4   * @brief Funzioni di utilità per l'incubatrice neonatale
5   *
6   * Implementazione delle funzioni di utilità per l'incubatrice neonatale.
7   */
8
9  #include "utils.h"
10 #include "error.h"
11
12 // ----- gestione della conversione da intero a 4 cifre a stringa -----
13 // -----
14 // -----
15
16 // stringa per overflow dei valori di temperatura e umidità
17 const char value_overflow[] PROGMEM = "xx.xx";
18
19 // buffer per conversione da float a stringa
20 char float_buffer[6];
21
22
23 // ----- calcolo dell'umidità relativa al raggiungimento del setpoint -----
24 // -----
25 // -----
26
27 // converte un dato (temperatura o umidità) in una stringa formattata
28 char *data_to_string(int16_t value) {
29     // controllo overflow per float negativi o superiori a 100
30     if (value < 0 || value > 9999) {
31         strcpy_P(float_buffer, value_overflow); // copia la stringa di overflow nel buffer
32         err_set(ERR_PRINT_OVERFLOW); // impostazione bit di errore
33     } else {
34         // suddivisione in cifre e scrittura nel buffer di output
35         float_buffer[5] = '\0'; // terminatore di fine stringa
36         float_buffer[4] = '0' + value % 10; value /= 10; // centesimi
37         float_buffer[3] = '0' + value % 10; value /= 10; // decimi
38         float_buffer[2] = '.'; // punto decimale
39         float_buffer[1] = '0' + value % 10; value /= 10; // unità
40         float_buffer[0] = (value == 0) ? ' ' : ('0' + value); // decine o spazio se 0
41     }
42
43     // restituzione del puntatore al buffer con la stringa formattata
44     return float_buffer;
45 }
46
47 // calcola l'umidità relativa al raggiungimento del setpoint di temperatura
48 int16_t rh_at_temp_setpoint(int16_t rh1, int16_t t1, int16_t t2) {
49     return round(rh1 * exp(428378.0 * (t1 - t2) / (24312.0 + t1) / (24312.0 + t2)));
50 }

```

B.3 Files del modulo per la gestione degli errori

Codice 5: File header con le funzioni per la gestione degli errori

```
1  /**
2  * @file error.h
3  * @author Giacomo Simonetto
4  * @brief Gestione degli errori dell'incubatrice neonatale
5  *
6  * Dichiarazione delle funzioni e dei flag necessari per il controllo degli
7  * errori dell'incubatrice neonatale.
8  */
9
10 #ifndef ERROR_H
11 #define ERROR_H
12
13 #include "config.h"
14
15 /**
16 * @brief Gestione degli errori dell'incubatrice neonatale.
17 *
18 * Gli errori si dividono in due categorie:
19 * - errori senza timer - non temporizzati:
20 *   errori che rimangono attivi fino a quando non vengono risolti, hanno
21 *   specifiche condizioni di attivazione e di disattivazione, ad esempio
22 *   gli errori di temperatura e umidità fuori dai range di sicurezza
23 *
24 * - errori con timer - temporizzati:
25 *   errori che vengono impostati quando si verifica un evento specifico
26 *   e rimangono attivi per uno specifico tempo limitato, dato da un timer
27 *   ad esempio gli errori di overflow nella stampa dei valori di temperatura
28 *   e umidità su display lcd e serial monitor
29 *
30 * Gli errori sono gestiti tramite un codice di stato a 16 bit, dove ogni bit
31 * rappresenta un errore specifico. I primi 10 bit sono riservati agli errori
32 * non temporizzati, mentre gli ultimi 6 bit sono riservati agli errori
33 * temporizzati. I bit sono definiti come segue:
34 *
35 * --- errori non temporizzati ---
36 *
37 * - bit0: temperatura troppo bassa (1)
38 *   la temperatura misurata dal sensore SHT20 è inferiore al range di
39 *   sicurezza impostato dal setpoint di temperatura e dalla soglia di
40 *   errore TEMP_ERR_THLD
41 *
42 * - bit1: temperatura troppo alta (2)
43 *   la temperatura misurata dal sensore SHT20 è superiore al range di
44 *   di sicurezza impostato dal setpoint di temperatura e dalla soglia
45 *   di errore TEMP_ERR_THLD
46 *
47 * - bit2: umidità troppo bassa (3)
48 *   l'umidità misurata dal sensore SHT20 è inferiore al range di
49 *   sicurezza impostato dal setpoint di umidità e dalla soglia di
50 *   di errore RH_ERR_THLD
51 *
52 * - bit3: umidità troppo alta (4)
53 *   l'umidità misurata dal sensore SHT20 è superiore al range di
54 *   sicurezza impostato dal setpoint di umidità e dalla soglia di
55 *   errore RH_ERR_THLD
56 *
57 * - bit4: overflow nella lettura della temperatura (5)
58 *   la temperatura misurata dal sensore SHT20 è minore di 0°C
59 *   o superiore a 100°C, assumerà il valore valido più vicino:
60 *   0°C se negativa o 99.99°C se maggiore di 100°C
61 *
62 * - bit5: overflow nella lettura dell'umidità (6)
63 *   l'umidità misurata dal sensore SHT20 è minore di 0% o maggiore
64 *   di 100%, assumerà il valore valido più vicino: 0% se negativa
65 *   o 99.99% se maggiore di 100%
66 *
67 * - bit6: non utilizzato (7)
68 *
69 * - bit7: non utilizzato (8)
70 *
71 * - bit8: non utilizzato (9)
72 *
73 * - bit9: non utilizzato (10)
```

```

74  *
75  *
76  * --- errori temporizzati ---
77  *
78  * - bit10: errore di comunicazione con il sensore SHT20 (11)
79  *      si sono verificati problemi di comunicazione con il sensore SHT20,
80  *      ad esempio quando il sensore non risponde alle richieste I2C
81  *
82  * - bit11: overflow nella stampa dei valori (12)
83  *      il valore di temperatura o umidità da stampare su display lcd
84  *      o serial monitor è minore di 0 o maggiore di 100, verrà stampata
85  *      la stringa "xx.xx" al posto del valore reale
86  *
87  * - bit12: non utilizzato (13)
88  *
89  * - bit13: non utilizzato (14)
90  *
91  * - bit14: non utilizzato (15)
92  *
93  * - bit15: non utilizzato (16)
94  */
95
96  // -----
97  // ----- definizione codici di errore -----
98  // -----
99
100 // codici di errore non temporizzati
101 #define ERR_LOW_TEMP uint8_t(0)      // temperatura troppo bassa
102 #define ERR_HIGH_TEMP uint8_t(1)     // temperatura troppo alta
103 #define ERR_LOW_RH uint8_t(2)        // umidità troppo bassa
104 #define ERR_HIGH_RH uint8_t(3)       // umidità troppo alta
105 #define ERR_TEMP_OVERFLOW uint8_t(4) // overflow nella lettura della temperatura
106 #define ERR_RH_OVERFLOW uint8_t(5)   // overflow nella lettura dell'umidità
107 #define ERR_UNUSED1 uint8_t(6)       // non utilizzato
108 #define ERR_UNUSED2 uint8_t(7)       // non utilizzato
109 #define ERR_UNUSED3 uint8_t(8)       // non utilizzato
110 #define ERR_UNUSED4 uint8_t(9)       // non utilizzato
111
112 // codici di errore temporizzati
113 #define ERR_SHT20 uint8_t(10)         // errore sensore SHT20
114 #define ERR_PRINT_OVERFLOW uint8_t(11) // overflow nella stampa dei valori
115 #define ERR_UNUSED5 uint8_t(12)      // non utilizzato
116 #define ERR_UNUSED6 uint8_t(13)      // non utilizzato
117 #define ERR_UNUSED7 uint8_t(14)      // non utilizzato
118 #define ERR_UNUSED8 uint8_t(15)      // non utilizzato
119
120
121 // -----
122 // ----- definizione timer errori -----
123 // -----
124
125 // array dei timer per gli errori temporizzati, in millisecondi
126 extern uint32_t err_timers[6];
127
128 // -----
129 // ----- dichiarazione funzioni errori -----
130 // -----
131
132 /**
133  * @brief Imposta un errore nel codice di stato.
134  *
135  * @param error_code flag del codice di errore da impostare
136  * @return true errore impostato correttamente
137  * @return false errore già presente, nessuna azione eseguita
138  */
139 void err_set(uint8_t error_code);
140
141 /**
142  * @brief Rimuove un errore dal codice di stato.
143  *
144  * @param error_code flag del codice di errore da rimuovere
145  * @return true errore rimosso correttamente
146  * @return false errore non presente, nessuna azione eseguita
147  */
148 void err_clear(uint8_t error_code);
149
150 /**
151  * @brief Verifica se un errore è presente nel codice di stato.
152  *

```

```

153  * @param error_code flag del codice di errore da verificare
154  * @return true errore presente nel codice di stato
155  * @return false errore non presente nel codice di stato
156  */
157  bool err_check(uint8_t error_code);
158
159  /**
160   * @brief Aggiorna lo stato del led di allarme.
161   *
162   * Il led di allarme viene acceso se è presente almeno un errore nel codice
163   * di stato, altrimenti viene spento.
164   */
165  void update_alarm_led();
166
167  /**
168   * @brief Verifica se ci sono errori temporizzati da rimuovere.
169   *
170   * Verifica ogni timer degli errori temporizzati presenti
171   */
172  void update_temporized_errors();
173
174  /**
175   * @brief Controlla se la temperatura misurata rientra nel range di sicurezza.
176   *
177   * Verifica se la temperatura misurata rientra nel range di sicurezza definito
178   * dal setpoint di temperatura e dalla soglia di errore TEMP_ERR_THLD.
179   */
180  void check_temp_range();
181
182  /**
183   * @brief Controlla se l'umidità misurata rientra nel range di sicurezza.
184   *
185   * Verifica se l'umidità misurata rientra nel range di sicurezza definito
186   * dal setpoint di umidità e dalla soglia di errore RH_ERR_THLD.
187   */
188  void check_rh_range();
189
190  #endif // ERROR_H

```

Codice 6: File sorgente con le funzioni per la gestione degli errori

```

1  /**
2   * @file error.cpp
3   * @author Giacomo Simonetto
4   * @brief Gestione degli errori dell'incubatrice neonatale
5   *
6   * Implementazione delle funzioni di controllo degli errori dell'incubatrice
7   * neonatale.
8   */
9
10 #include "error.h"
11 #include "utils.h"
12
13 // timer per gli errori temporizzati, in millisecondi
14 uint32_t err_timers[6] = {0, 0, 0, 0, 0, 0};
15
16 // imposta un errore nel codice di stato
17 void err_set(uint8_t error_code) {
18     // verifica validità del codice di errore (0-15)
19     if (error_code > 15)
20         return;
21
22     // imposta l'errore nel codice di stato
23     status.error_code |= (1U << error_code);
24
25     // imposta il timer per gli errori temporizzati
26     if (error_code >= 10)
27         err_timers[error_code - 10] = millis();
28 }
29
30 // rimuove un errore dal codice di stato
31 void err_clear(uint8_t error_code) {
32     // verifica validità del codice di errore (0-15)
33     if (error_code > 15)
34         return;

```

```

35 // rimuove l'errore dal codice di stato
36 status.error_code &= ~(1U << error_code);
37 }
38
39 // verifica se un errore è presente nel codice di stato
40 bool err_check(uint8_t error_code) {
41     // verifica validità del codice di errore (0-15)
42     if (error_code > 15)
43         return false;
44
45     // verifica se l'errore è presente nel codice di stato
46     return (status.error_code & (1U << error_code)) != 0;
47 }
48
49 // aggiorna lo stato del led di allarme
50 void update_alarm_led() {
51     if (!status.alarm_led && status.error_code != 0) {
52         digitalWrite(ALARM_LED, LED_ON);
53         status.alarm_led = true;
54     }
55     else if (status.alarm_led && status.error_code == 0) {
56         digitalWrite(ALARM_LED, LED_OFF);
57         status.alarm_led = false;
58     }
59 }
60
61 // verifica se ci sono errori temporizzati da rimuovere
62 void update_temporized_errors() {
63     for (uint8_t i = 10; i < 16; i++) {
64         if (err_check(i) && millis() - err_timers[i - 10] >= ERR_TIMER) {
65             err_timers[i - 10] = 0; // resetta il timer
66             err_clear(i); // rimuove l'errore temporizzato
67         }
68     }
69 }
70
71 // verifica se la temperatura misurata rientra nel range di sicurezza
72 void check_temp_range() {
73     // temperatura troppo bassa
74     if (!err_check(ERR_LOW_TEMP) && status.temp_sht20 < status.temp_setpoint - TEMP_ERR_THLD -
75         ↪ TEMP_ERR_HYST)
76         err_set(ERR_LOW_TEMP); // errore temperatura troppo bassa
77     else if (err_check(ERR_LOW_TEMP) && status.temp_sht20 >= status.temp_setpoint - TEMP_ERR_THLD +
78         ↪ TEMP_ERR_HYST)
79         err_clear(ERR_LOW_TEMP); // rimozione errore temperatura troppo bassa
80
81     // temperatura troppo alta
82     else if (!err_check(ERR_HIGH_TEMP) && status.temp_sht20 > status.temp_setpoint + TEMP_ERR_THLD +
83         ↪ TEMP_ERR_HYST)
84         err_set(ERR_HIGH_TEMP); // errore temperatura troppo alta
85     else if (err_check(ERR_HIGH_TEMP) && status.temp_sht20 <= status.temp_setpoint + TEMP_ERR_THLD -
86         ↪ TEMP_ERR_HYST)
87         err_clear(ERR_HIGH_TEMP); // rimozione errore temperatura troppo alta
88 }
89
90 // verifica se l'umidità misurata rientra nel range di sicurezza
91 void check_rh_range() {
92     // umidità troppo bassa
93     if (!err_check(ERR_LOW_RH) && status.rh_sht20 < status.rh_setpoint - RH_ERR_THLD - RH_ERR_HYST)
94         err_set(ERR_LOW_RH); // errore umidità troppo bassa
95     else if (err_check(ERR_LOW_RH) && status.rh_sht20 >= status.rh_setpoint - RH_ERR_THLD + RH_ERR_HYST)
96         err_clear(ERR_LOW_RH); // rimozione errore umidità troppo bassa
97
98     // umidità troppo alta
99     else if (!err_check(ERR_HIGH_RH) && status.rh_sht20 > status.rh_setpoint + RH_ERR_THLD + RH_ERR_HYST)
100         err_set(ERR_HIGH_RH); // errore umidità troppo alta
101     else if (err_check(ERR_HIGH_RH) && status.rh_sht20 <= status.rh_setpoint + RH_ERR_THLD - RH_ERR_HYST)
102         err_clear(ERR_HIGH_RH); // rimozione errore umidità troppo alta
103 }

```

B.4 Files del modulo per il controllo attuatori

Codice 7: File header con le funzioni per il controllo degli attuatori

```
1  /**
2   * @file actuators.h
3   * @author Giacomo Simonetto
4   * @brief Controllo degli attuatori dell'incubatrice neonatale
5   *
6   * Dichiarazione delle funzioni di controllo degli attuatori dell'incubatrice
7   * neonatale e dei relativi led di stato del pannello frontale.
8   */
9
10 #ifndef ACTUATORS_H
11 #define ACTUATORS_H
12
13 #include "config.h"
14
15 /**
16  * @brief Accensione del riscaldatore.
17  *
18  * Questa funzione accende il riscaldatore e il led di stato corrispondente.
19  * Accende anche la ventola ausiliaria se è spenta.
20  *
21  * @return true riscaldatore acceso con successo
22  * @return false riscaldatore già acceso, nessuna azione eseguita
23  */
24 bool heat_turn_on();
25
26 /**
27  * @brief Spegnimento del riscaldatore.
28  *
29  * Questa funzione spegne il riscaldatore e il led di stato corrispondente.
30  * Spegne anche la ventola ausiliaria se l'umidificatore è spento.
31  *
32  * @return true riscaldatore spento con successo
33  * @return false riscaldatore già spento, nessuna azione eseguita
34  */
35 bool heat_turn_off();
36
37 /**
38  * @brief Accensione dell'umidificatore.
39  *
40  * Questa funzione accende l'umidificatore e il led di stato corrispondente.
41  * Accende anche la ventola ausiliaria se è spenta.
42  *
43  * Se è necessario fare un refill dell'acqua, l'umidificatore non viene acceso
44  * e la funzione restituisce false senza fare nulla.
45  *
46  * All'accensione dell'umidificatore, viene aggiornato il timer che tiene
47  * traccia dell'ultima accensione dell'umidificatore, per poter gestire
48  * il timer di refill dell'acqua.
49  *
50  * @return true umidificatore acceso con successo
51  * @return false umidificatore già acceso, nessuna azione eseguita
52  */
53 bool rh_turn_on();
54
55 /**
56  * @brief Spegnimento dell'umidificatore.
57  *
58  * Questa funzione spegne l'umidificatore e il led di stato corrispondente.
59  * Spegne anche la ventola ausiliaria se il riscaldatore è spento.
60  *
61  * Aggiorna il timer di refill dell'acqua, aggiungendo il tempo trascorso
62  * dall'ultima accensione dell'umidificatore.
63  *
64  * @return true umidificatore spento con successo
65  * @return false umidificatore già spento, nessuna azione eseguita
66  */
67 bool rh_turn_off();
68
69 /**
70  * @brief Accensione dell'illuminazione.
71  *
72  * @return true illuminazione accesa con successo
73  * @return false illuminazione già accesa, nessuna azione eseguita
```

```

74  */
75  bool light_turn_on();
76
77  /**
78   * @brief Spegnimento dell'illuminazione.
79   *
80   * @return true illuminazione spenta con successo
81   * @return false illuminazione già spenta, nessuna azione eseguita
82   */
83  bool light_turn_off();
84
85  #endif // ACTUATORS_H

```

Codice 8: File sorgente con le funzioni per il controllo degli attuatori

```

1  /**
2   * @file actuators.cpp
3   * @author Giacomo Simonetto
4   * @brief Controllo degli attuatori dell'incubatrice neonatale
5   *
6   * Implementazione delle funzioni di controllo degli attuatori dell'incubatrice
7   * neonatale e dei relativi led di stato del pannello frontale.
8   */
9
10 #include "actuators.h"
11 #include "utils.h"
12
13 // accensione riscaldatore
14 bool heat_turn_on() {
15     // se il riscaldatore è già acceso, restituisce false senza fare nulla
16     if (status.heat_relay)
17         return false;
18
19     // accensione riscaldatore e LED di stato
20     digitalWrite(HEAT_RELAY, RELAY_ON); // accensione riscaldatore
21     digitalWrite(HEAT_LED, LED_ON);     // accensione LED riscaldatore
22     status.heat_relay = true; // aggiornamento stato riscaldatore
23
24     // accensione ventola ausiliaria se spenta
25     if (!status.fan_relay) {
26         digitalWrite(FAN_RELAY, RELAY_ON); // accensione ventola ausiliaria
27         status.fan_relay = true; // aggiornamento stato ventola ausiliaria
28     }
29
30     // accensione avvenuta con successo
31     return true;
32 }
33
34 // spegnimento riscaldatore
35 bool heat_turn_off() {
36     // se il riscaldatore è già spento, restituisce false senza fare nulla
37     if (!status.heat_relay)
38         return false;
39
40     // spegnimento riscaldatore e LED di stato
41     digitalWrite(HEAT_RELAY, RELAY_OFF); // spegnimento riscaldatore
42     digitalWrite(HEAT_LED, LED_OFF);     // spegnimento LED riscaldatore
43     status.heat_relay = false; // aggiornamento stato riscaldatore
44
45     // spegnimento ventola ausiliaria se l'umidificatore è spento
46     if (!status.rh_relay) {
47         digitalWrite(FAN_RELAY, RELAY_OFF); // spegnimento ventola ausiliaria
48         status.fan_relay = false; // aggiornamento stato ventola ausiliaria
49     }
50
51     // spegnimento avvenuto con successo
52     return true;
53 }
54
55 // accensione umidificatore
56 bool rh_turn_on() {
57     // se l'umidificatore è già acceso o serve effettuare il refill dell'acqua,
58     // restituisce false senza fare nulla
59     if (status.rh_relay || status.refill_led)
60         return false;

```

```

61
62 // accensione umidificatore e LED di stato
63 digitalWrite(RH_RELAY, RELAY_ON); // accensione umidificatore
64 digitalWrite(RH_LED, LED_ON); // accensione LED umidificatore
65 status.rh_relay = true; // aggiornamento stato umidificatore
66
67 // accensione ventola ausiliaria se spenta
68 if (!status.fan_relay) {
69     digitalWrite(FAN_RELAY, RELAY_ON); // accensione ventola ausiliaria
70     status.fan_relay = true; // aggiornamento stato ventola ausiliaria
71 }
72
73 // gestione contatore refill
74 timers.last_rh_on = millis();
75
76 // accensione avvenuta con successo
77 return true;
78 }
79
80 // spegnimento umidificatore
81 bool rh_turn_off() {
82     // se l'umidificatore è già spento, restituisce false senza fare nulla
83     if (!status.rh_relay)
84         return false;
85
86     // spegnimento umidificatore e LED di stato
87     digitalWrite(RH_RELAY, RELAY_OFF); // spegnimento umidificatore
88     digitalWrite(RH_LED, LED_OFF); // spegnimento LED umidificatore
89     status.rh_relay = false; // aggiornamento stato umidificatore
90
91     // spegnimento ventola ausiliaria se il riscaldatore è spento
92     if (!status.heat_relay) {
93         digitalWrite(FAN_RELAY, RELAY_OFF); // spegnimento ventola ausiliaria
94         status.fan_relay = false; // aggiornamento stato ventola ausiliaria
95     }
96
97     // gestione contatore refill
98     timers.refill_counter += millis() - timers.last_rh_on;
99
100 // spegnimento avvenuto con successo
101 return true;
102 }
103
104 // accensione illuminazione
105 bool light_turn_on() {
106     // se l'illuminazione è già accesa, restituisce false senza fare nulla
107     if (status.light_relay)
108         return false;
109
110     digitalWrite(LIGHT_RELAY, RELAY_ON); // accensione illuminazione
111     status.light_relay = true; // aggiornamento stato illuminazione
112
113     // accensione avvenuta con successo
114     return true;
115 }
116
117 // spegnimento illuminazione
118 bool light_turn_off() {
119     // se l'illuminazione è già spenta, restituisce false senza fare nulla
120     if (!status.light_relay)
121         return false;
122
123     digitalWrite(LIGHT_RELAY, RELAY_OFF); // spegnimento illuminazione
124     status.light_relay = false; // aggiornamento stato illuminazione
125
126     // spegnimento avvenuto con successo
127     return true;
128 }

```

B.5 Files del modulo per la gestione del display lcd

Codice 9: File header con le funzioni per la gestione del display lcd

```
1  /**
2   * @file lcd.h
3   * @author Giacomo Simonetto
4   * @brief Gestione del display lcd dell'incubatrice neonatale
5   *
6   * Dichiarazione delle funzioni per la gestione del display lcd del pannello
7   * frontale dell'incubatrice neonatale.
8   */
9
10 #ifndef LCD_H
11 #define LCD_H
12
13 #include "config.h"
14
15 /**
16  * @brief Stampa il messaggio di avvio sul display lcd.
17  */
18 void lcd_boot_message();
19
20 /**
21  * @brief Stampa lo stato dell'incubatrice su display lcd.
22  *
23  * Questa funzione stampa sul display lcd:
24  * - la temperatura misurata e il relativo setpoint
25  * - l'umidità misurata e il relativo setpoint
26  * - stato del controllo degli attuatori (manuale o automatico)
27  * - il codice di eventuali errori che si sono verificati
28  */
29 void lcd_print_status();
30
31 /**
32  * @brief Accensione manuale del riscaldatore.
33  */
34 void lcd_man_heat_on();
35
36 /**
37  * @brief Spegnimento manuale del riscaldatore.
38  */
39 void lcd_man_heat_off();
40
41 /**
42  * @brief Accensione manuale dell'umidificatore.
43  */
44 void lcd_man_rh_on();
45
46 /**
47  * @brief Spegnimento manuale dell'umidificatore.
48  */
49 void lcd_man_rh_off();
50
51 /**
52  * @brief Accensione manuale dell'illuminazione.
53  */
54 void lcd_man_light_on();
55
56 /**
57  * @brief Spegnimento manuale dell'illuminazione.
58  */
59 void lcd_man_light_off();
60
61 /**
62  * @brief Visualizzazione del nuovo setpoint di temperatura.
63  */
64 void lcd_new_temp_setpoint(float new_temp_setpoint);
65
66 /**
67  * @brief Visualizzazione del nuovo setpoint di umidità.
68  */
69 void lcd_new_rh_setpoint(float new_rh_setpoint);
70
71 /**
72  * @brief Stampa il messaggio di refill sul display lcd.
73  */
```

```

74 void lcd_refill_message();
75
76 /**
77  * @brief Stampa il messaggio di errore associato al codice di errore
78  * restituito dal sensore SHT20.
79  *
80  * @param errorCode codice di errore restituito dal sensore SHT20
81  */
82 void lcd_sht20_error(int errorCode);
83
84 #endif // LCD_H

```

Codice 10: File sorgente con le funzioni per la gestione del display lcd

```

1  /**
2  * @file lcd.cpp
3  * @author Giacomo Simonetto
4  * @brief Gestione del display lcd dell'incubatrice neonatale
5  *
6  * Implementazione delle funzioni per la gestione del display lcd del pannello
7  * frontale dell'incubatrice neonatale.
8  */
9
10 #include "lcd.h"
11 #include "utils.h"
12
13 /**
14  * @brief Controllo del display lcd.
15  *
16  * Di seguito sono riportate le tecniche utilizzate per la gestione del
17  * display lcd adottate per evitare flickering e aggiornamenti troppo
18  * frequenti delle informazioni stampate sul display.
19  *
20  *
21  * ----- gestione in pagine e temporizzazione delle pagine -----
22  *
23  * Le informazioni stampate sul display sono suddivise in pagine, ciascuna con
24  * il proprio indice. In questo modo è possibile sapere cosa è stato stampato
25  * sul display. Le pagine sono le seguenti:
26  *
27  * 1. LCD_DATA_PAGE: mostra temperatura, umidità relativi setpoint, stato
28  * del controllo attuatori ed eventuali codici di errore
29  *
30  * 2. LCD_TEMP_SP_PAGE: mostra l'aggiornamento del setpoint di temperatura
31  *
32  * 3. LCD_RH_SP_PAGE: mostra l'aggiornamento del setpoint di umidità
33  *
34  * 4. LCD_MESSAGE_PAGE: mostra messaggi di accensione, spegnimento e refill
35  * degli attuatori e messaggi di errore del sensore SHT20
36  *
37  * Ad ogni aggiornamento del display viene aggiornato il timer lcd_update,
38  * con l'istante dell'ultima stampa e l'indice di pagina lcd_page in base
39  * al contenuto stampato sul display.
40  *
41  * La pagina relativa alla visualizzazione dei dati (LCD_DATA_PAGE) viene
42  * aggiornata solo dopo LCD_UPDATE_PERIOD millisecondi dall'ultima stampa.
43  * In questo modo l'utente ha il tempo di leggere le informazioni stampate
44  * precedentemente sul display lcd, prima che vengano sovrascritte dalle
45  * nuove informazioni.
46  *
47  * Tutte le restanti pagine non hanno temporizzazione, per cui le nuove
48  * informazioni vengono subito stampate sul display lcd sovrascrivendo quelle
49  * precedentemente stampate. Questo comportamento è stato scelto perché le
50  * informazioni stampate su queste pagine coincidono con eventi specifici
51  * che richiedono un aggiornamento immediato del display, come ad esempio
52  * l'accensione o lo spegnimento di un attuatore o la visualizzazione di un
53  * messaggio di errore del sensore SHT20.
54  *
55  * NOTA: il messaggio di avvio non ha temporizzazione, viene mostrato solo
56  * durante la fase di avvio e inizializzazione dell'incubatrice.
57  *
58  *
59  * ----- gestione dei flickering per rapido aggiornamento -----
60  *
61  * Per evitare flickering dovuti a frequenti e continui aggiornamenti di tutte

```

```

62 * le informazioni stampate sul display, è stato scelto di tenere traccia di
63 * ciò che è stato precedentemente stampato e aggiornare selettivamente solo
64 * i dati modificati.
65 *
66 * Di fronte ad un cambio di pagina, il display viene completamente aggiornato
67 * con le nuove informazioni. Viceversa, se la nuova pagina da stampare è la
68 * stessa attualmente stampata, si aggiorneranno solo i dati modificati.
69 *
70 * Per memorizzare i valori dei dati precedentemente stampati si utilizzano
71 * delle apposite variabili che sono mantenute aggiornate all'ultimo valore
72 * stampato sul display. Di seguito sono elencati i dati variabili di ogni
73 * pagina e le relative variabili che ne tengono traccia:
74 *
75 * - LCD_DATA_PAGE:
76 *   - temperatura ed umidità misurate con relativi setpoint tracciati dalle
77 *     variabili old_temp, old_rh, old_temp_setpoint e old_rh_setpoint
78 *   - codice di errore tracciato dalla variabile old_error_code
79 *
80 * - LCD_TEMP_SETPOINT_PAGE e LCD_RH_SETPOINT_PAGE:
81 *   ad ogni chiamata delle funzioni di aggiornamento del setpoint corrisponde
82 *   ad un aggiornamento del setpoint, quindi andrà sempre aggiornato e non
83 *   è necessario memorizzare il valore precedentemente stampato
84 *
85 * - LCD_MESSAGE_PAGE:
86 *   non contengono dati, per cui andrà sempre aggiornata l'intera pagina
87 *
88 *
89 * ----- gestione dei messaggi di errore del sensore SHT20 -----
90 *
91 * I messaggi di errore del sensore SHT20 sono salvati sotto forma di struct
92 * lcd_message formate di 4 stringhe di 21 caratteri ciascuna, che permettono
93 * di salvare il testo suddiviso nelle 4 righe del display lcd.
94 *
95 * Siccome i codici di errore del sensore SHT20 vanno da 0x80 a 0x8A, è stato
96 * scelto di rappresentare le struct con le descrizioni in un array tale per
97 * cui la struct all'indice "i" corrisponde al codice di errore "0x80 +i".
98 * In questo modo risulta immediato trovare il messaggio di errore da stampare
99 * sul display lcd in base al codice di errore restituito dal sensore SHT20.
100 */
101
102 // -----
103 // ----- variabili e flag per la paginazione -----
104 // -----
105
106 uint8_t lcd_page = 0;
107
108 #define LCD_DATA_PAGE 1
109 #define LCD_TEMP_SETPOINT_PAGE 2
110 #define LCD_RH_SETPOINT_PAGE 3
111 #define LCD_MESSAGE_PAGE 4
112
113 // -----
114 // ----- variabili per memorizzare dati stampati -----
115 // -----
116
117 int16_t old_temp = 0U;
118 int16_t old_rh = 0U;
119 int16_t old_temp_setpoint = 0U;
120 int16_t old_rh_setpoint = 0U;
121 uint16_t old_error_code = 0U;
122
123 // -----
124 // ----- messaggi di errore del sensore SHT20 -----
125 // -----
126
127 // struct per memorizzare i messaggi da stampare sul display lcd
128 struct lcd_message {
129     char line1[21]; // prima riga del messaggio
130     char line2[21]; // seconda riga del messaggio
131     char line3[21]; // terza riga del messaggio
132     char line4[21]; // quarta riga del messaggio
133 };
134
135 // messaggi di errore del sensore SHT20 da stampare sul display lcd
136 const struct lcd_message sht20_error_lcd_message[11] PROGMEM = {
137     {" SHT20 Error: ", " Error during I2C ", " write command ", "

```

```

141     {" SHT20 Error: ", " Error during I2C ", " read bytes ", " },
142     {" SHT20 Error: ", " Failed to switch ", " off the internal ", " heater ", " },
143     {" SHT20 Error: ", "Sensor not connected", " or does not ack ", " on the I2C bus ", " },
144     {" SHT20 Error: ", " CRC check failed ", "for the temperature ", " reading ", " },
145     {" SHT20 Error: ", " CRC check failed ", " for the humidity ", " reading ", " },
146     {" SHT20 Error: ", " CRC check failed ", " for the status ", " register ", " },
147     {" SHT20 Error: ", " Heater is in its ", "mandatory cool-down ", " period ", " },
148     {" SHT20 Error: ", " Failed to switch ", " on the internal ", " heater ", " },
149     {" SHT20 Error: ", " Invalid resolution ", " parameter provided ", " },
150     {" SHT20 Error: ", " Unknown error ", " occurred ", " },
151 };
152
153
154 // ----- funzioni per la gestione del display -----
155 // -----
156
157 // stampa messaggio di avvio sul display lcd
158 void lcd_boot_message() {
159     lcd.clear();
160     lcd.setCursor(0, 0);
161     lcd.print(F(" Neonatal incubator "));
162     lcd.setCursor(0, 1);
163     lcd.print(F(" prototype "));
164     lcd.setCursor(0, 3);
165     lcd.print(F("Navlab - DEI - UNIPD"));
166 }
167
168
169 // stampa lo stato dell'incubatrice sul display lcd
170 void lcd_print_status() {
171     // timer per visualizzazione messaggi sul display
172     if (lcd_page != LCD_DATA_PAGE && millis() - timers.lcd_update < LCD_UPDATE_PERIOD)
173         return;
174
175     /*
176     * Mappa dei dati stampati sul display lcd per facilitare
177     * l'individuazione delle relative posizioni.
178     *
179     * | 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 |
180     * | 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 |
181     * - / - - - - - - - - - - - - - - - - /
182     * 0 | T : x x . x x ° C / x x . x x ° C |
183     * 1 | H : x x . x x % / x x . x x % |
184     * 2 | C t r l s t a t u s : M A N U A L | per controllo manuale
185     * 3 | C t r l s t a t u s : A U T O | per controllo automatico
186     * 3 | N o e r r o r s d e t e c t e d | in assenza di errori
187     * 3 | E r r o r c o d e : x x x x | in presenza di errori
188     * - / - - - - - - - - - - - - - - - - /
189     */
190
191     // pulizia e stampa intestazioni per cambio pagina
192     if (lcd_page != LCD_DATA_PAGE) {
193         lcd.clear();
194         lcd.setCursor(0, 0);
195         lcd.print(F("T:"));
196         lcd.setCursor(8, 0);
197         lcd.print((char)223);
198         lcd.print(F("C / "));
199         lcd.setCursor(18, 0);
200         lcd.print((char)223);
201         lcd.print('C');
202         lcd.setCursor(0, 1);
203         lcd.print(F("H:"));
204         lcd.setCursor(8, 1);
205         lcd.print(F("% / "));
206         lcd.setCursor(18, 1);
207         lcd.print('%');
208         lcd.setCursor(0, 2);
209         lcd.print(F("Ctrl status: "));
210         lcd.print(status.manual_ctrl ? F("MAN ") : F("AUTO"));
211     }
212
213     // aggiornamento temperatura
214     if (lcd_page != LCD_DATA_PAGE || status.temp_sht20 != old_temp) {
215         lcd.setCursor(3, 0);
216         lcd.print(data_to_string(status.temp_sht20));
217         old_temp = status.temp_sht20;
218     }
219 }

```

```

220
221 // aggiornamento setpoint di temperatura
222 if (lcd_page != LCD_DATA_PAGE || status.temp_setpoint != old_temp_setpoint) {
223     lcd.setCursor(13, 0);
224     lcd.print(data_to_string(status.temp_setpoint));
225     old_temp_setpoint = status.temp_setpoint;
226 }
227
228 // aggiornamento umidità
229 if (lcd_page != LCD_DATA_PAGE || status.rh_sht20 != old_rh) {
230     lcd.setCursor(3, 1);
231     lcd.print(data_to_string(status.rh_sht20));
232     old_rh = status.rh_sht20;
233 }
234
235 // aggiornamento setpoint di umidità
236 if (lcd_page != LCD_DATA_PAGE || status.rh_setpoint != old_rh_setpoint) {
237     lcd.setCursor(13, 1);
238     lcd.print(data_to_string(status.rh_setpoint));
239     old_rh_setpoint = status.rh_setpoint;
240 }
241
242 // aggiornamento errori
243 if (lcd_page != LCD_DATA_PAGE || status.error_code != old_error_code) {
244     // pulizia della riga dei messaggi di errore
245     lcd.setCursor(0, 3);
246     lcd.print(F(""));
247
248     // stampa codice di errore se presente, altrimenti messaggio di assenza errori
249     lcd.setCursor(0, 3);
250     if (!status.error_code) {
251         lcd.print(F("No errors detected "));
252     } else {
253         lcd.print("Error code: ");
254         lcd.print(status.error_code);
255     }
256
257     // aggiornamento variabile di stato
258     old_error_code = status.error_code;
259 }
260
261 lcd_page = LCD_DATA_PAGE; // aggiornamento pagina
262 timers.lcd_update = millis(); // aggiornamento timer
263 }
264
265 // visualizzazione del nuovo setpoint di temperatura
266 void lcd_new_temp_setpoint(float new_temp_setpoint) {
267     // pulizia e stampa intestazioni per cambio pagina
268     if (lcd_page != LCD_TEMP_SETPOINT_PAGE) {
269         lcd.clear();
270         lcd.setCursor(0, 0);
271         lcd.print(F("Nuovo setpoint temp:"));
272         lcd.setCursor(11, 1);
273         lcd.print((char)223);
274         lcd.print('C');
275         lcd.setCursor(0, 3);
276         lcd.print(F("precedente: "));
277         lcd.print(data_to_string(status.temp_setpoint));
278         lcd.print((char)223);
279         lcd.print('C');
280     }
281
282     // stampa nuovo setpoint temperatura
283     if (lcd_page != LCD_TEMP_SETPOINT_PAGE || new_temp_setpoint != old_temp_setpoint) {
284         lcd.setCursor(6, 1);
285         lcd.print(data_to_string(new_temp_setpoint));
286         old_temp_setpoint = new_temp_setpoint;
287     }
288
289     lcd_page = LCD_TEMP_SETPOINT_PAGE; // aggiornamento pagina
290     timers.lcd_update = millis(); // aggiornamento timer
291 }
292
293 // visualizzazione del nuovo setpoint di umidità
294 void lcd_new_rh_setpoint(float new_rh_setpoint) {
295     // pulizia e stampa intestazioni per cambio pagina
296     if (lcd_page != LCD_RH_SETPOINT_PAGE) {
297         lcd.clear();
298         lcd.setCursor(0, 0);

```

```

299     lcd.print(F("Nuovo setpoint RH:"));
300     lcd.setCursor(11, 1);
301     lcd.print('%');
302     lcd.setCursor(0, 3);
303     lcd.print(F("precedente: "));
304     lcd.print(data_to_string(status.rh_setpoint));
305     lcd.print('%');
306 }
307
308 // aggiornamento setpoint di umidità
309 if (lcd_page != LCD_RH_SETPOINT_PAGE || new_rh_setpoint != old_rh_setpoint) {
310     lcd.setCursor(6, 1);
311     lcd.print(data_to_string(new_rh_setpoint));
312     old_rh_setpoint = new_rh_setpoint;
313 }
314
315 lcd_page = LCD_RH_SETPOINT_PAGE; // aggiornamento pagina
316 timers.lcd_update = millis(); // aggiornamento timer
317 }
318
319 // accensione manuale del riscaldatore
320 void lcd_man_heat_on() {
321     lcd.clear();
322     lcd.setCursor(0, 1);
323     lcd.print(F("Accensione manuale"));
324     lcd.setCursor(0, 2);
325     lcd.print(F("del riscaldatore!"));
326
327     lcd_page = LCD_MESSAGE_PAGE; // aggiornamento pagina
328     timers.lcd_update = millis(); // aggiornamento timer
329 }
330
331 // spegnimento manuale del riscaldatore
332 void lcd_man_heat_off() {
333     lcd.clear();
334     lcd.setCursor(0, 1);
335     lcd.print(F("Spegnimento manuale"));
336     lcd.setCursor(0, 2);
337     lcd.print(F("del riscaldatore!"));
338
339     lcd_page = LCD_MESSAGE_PAGE; // aggiornamento pagina
340     timers.lcd_update = millis(); // aggiornamento timer
341 }
342
343 // accensione manuale dell'umidificatore
344 void lcd_man_rh_on() {
345     lcd.clear();
346     lcd.setCursor(0, 1);
347     lcd.print(F("Accensione manuale"));
348     lcd.setCursor(0, 2);
349     lcd.print(F("dell'umidificatore!"));
350
351     lcd_page = LCD_MESSAGE_PAGE; // aggiornamento pagina
352     timers.lcd_update = millis(); // aggiornamento timer
353 }
354
355 // spegnimento manuale dell'umidificatore
356 void lcd_man_rh_off() {
357     lcd.clear();
358     lcd.setCursor(0, 1);
359     lcd.print(F("Spegnimento manuale"));
360     lcd.setCursor(0, 2);
361     lcd.print(F("dell'umidificatore!"));
362
363     lcd_page = LCD_MESSAGE_PAGE; // aggiornamento pagina
364     timers.lcd_update = millis(); // aggiornamento timer
365 }
366
367 // accensione manuale dell'illuminazione
368 void lcd_man_light_on() {
369     lcd.clear();
370     lcd.setCursor(0, 1);
371     lcd.print(F("Accensione manuale"));
372     lcd.setCursor(0, 2);
373     lcd.print(F("dell'illuminazione!"));
374
375     lcd_page = LCD_MESSAGE_PAGE; // aggiornamento pagina
376     timers.lcd_update = millis(); // aggiornamento timer
377 }

```

```

378
379 // spegnimento manuale dell'illuminazione
380 void lcd_man_light_off() {
381     lcd.clear();
382     lcd.setCursor(0, 1);
383     lcd.print(F("Spegnimento manuale"));
384     lcd.setCursor(0, 2);
385     lcd.print(F("dell'illuminazione!"));
386
387     lcd_page = LCD_MESSAGE_PAGE; // aggiornamento pagina
388     timers.lcd_update = millis(); // aggiornamento timer
389 }
390
391 // stampa il messaggio di refill sul display lcd
392 void lcd_refill_message() {
393     lcd.clear();
394     lcd.setCursor(0, 1);
395     lcd.print(F("Ricaricare l'acqua"));
396     lcd.setCursor(0, 2);
397     lcd.print(F("nell'umidificatore!"));
398
399     lcd_page = LCD_MESSAGE_PAGE; // aggiornamento pagina
400     timers.lcd_update = millis(); // aggiornamento timer
401 }
402
403 // stampa il messaggio di errore del sensore SHT20
404 void lcd_sht20_error(int errorCode) {
405     if (errorCode < 0x81 || errorCode > 0x8B)
406         errorCode = 0x8B; // errore sconosciuto
407
408     // recupero del messaggio dalla memoria flash
409     struct lcd_message message;
410     memcpy_P(&message, &sht20_error_lcd_message[errorCode - 0x81], sizeof(struct lcd_message));
411
412     // stampa del messaggio sul display
413     lcd.clear();
414     lcd.setCursor(0, 0);
415     lcd.print(message.line1);
416     lcd.setCursor(0, 1);
417     lcd.print(message.line2);
418     lcd.setCursor(0, 2);
419     lcd.print(message.line3);
420     lcd.setCursor(0, 3);
421     lcd.print(message.line4);
422
423     lcd_page = LCD_MESSAGE_PAGE; // aggiornamento pagina
424     timers.lcd_update = millis(); // aggiornamento timer
425 }

```

B.6 Files del modulo per la gestione del sensore di temperatura e umidità SHT20

Codice 11: File header con le funzioni per la gestione del sensore SHT20

```
1  /**
2   * @file sht20.h
3   * @author Giacomo Simonetto
4   * @brief Gestione del sensore SHT20
5   *
6   * Dichiarazione delle funzioni per la gestione del sensore SHT20.
7   */
8
9  #ifndef SHT20_H
10 #define SHT20_H
11
12 #include <Arduino.h>
13 #include <SHT2x.h>
14
15 /**
16  * @brief Classe per la gestione non bloccante del sensore SHT20.
17  *
18  * Per un corretto funzionamento, è necessario invocare la funzione update().
19  * ad ogni iterazione del loop principale. Tale funzione gestisce in maniera
20  * automatica le operazioni da eseguire in base allo stato del sensore.
21  *
22  * Al termine di ogni invocazione di update() in base al valore restituito
23  * da tale funzione, si consigliano le seguenti azioni:
24  *
25  * - false: non sono disponibili nuovi dati o si è verificato un errore,
26  *          è consigliato verificare il codice di errore tramite get_error()
27  *
28  * - true: sono disponibili nuovi dati per la temperatura e l'umidità,
29  *         procedere con la lettura di entrambi i valori tramite le funzioni
30  *         get_temperature() e get_humidity() il prima possibile, non verrà
31  *         effettuata la richiesta di nuovi dati finché quelli disponibili
32  *         non saranno stati letti
33  *
34  * Per un corretto funzionamento del timer di lettura del sensore SHT20, è
35  * necessario resettare il timer di lettura del sensore tramite la funzione
36  * init_timer() prima della prima lettura del sensore o se il sensore non è
37  * stato utilizzato per un periodo di tempo superiore al periodo di lettura
38  * impostato. La prima lettura successiva avverrà dopo il periodo di lettura
39  * impostato dall'invocazione di init_timer().
40  */
41 class sht20 {
42 public:
43     /**
44      * @brief Costruttore della classe sht20.
45      *
46      * Riceve come parametro il periodo di lettura del sensore SHT20, espresso
47      * in millisecondi. Di default viene impostato a 1000 ms (1 secondo).
48      *
49      * Dal datasheet è sconsigliato scendere sotto al secondo per evitare
50      * fenomeni di self-heating del sensore.
51      *
52      * @param read_period periodo di lettura del sensore
53      */
54     sht20(uint32_t read_period = 1000);
55
56     /**
57      * @brief Inizializzazione del sensore SHT20.
58      *
59      * @return true se l'inizializzazione è andata a buon fine
60      * @return false se si è verificato un errore
61      */
62     bool begin();
63
64     /**
65      * @brief Inizializza il timer per la lettura del sensore SHT20.
66      *
67      * La funzione inizializza il timer per la lettura del sensore SHT20,
68      * in modo da eliminare eventuali offset temporali precedenti alla prima
69      * lettura o dovuti ad un periodo di inattività del sensore superiore al
70      * periodo di lettura impostato.
```

```

71      *
72      * La lettura successiva avverrà dopo il periodo di lettura impostato
73      * dall'invocazione di questa funzione.
74      */
75      void init_timer() {
76          read_timer = millis();
77      }
78
79      /**
80      * @brief Effettua l'aggiornamento dello stato del sensore SHT20.
81      *
82      * La funzione gestisce in autonomia le operazioni da eseguire in base
83      * allo stato del sensore, per cui va invocata ad ogni iterazione del
84      * loop principale.
85      *
86      * Se non sono disponibili nuovi dati per la temperatura e l'umidità, la
87      * funzione restituisce false ed è possibile verificare se si è verificato
88      * un errore invocando la funzione get_error(). L'errore viene resettato
89      * a 0 al successivo aggiornamento con successo dello stato del sensore,
90      * per cui è necessario leggere il codice di errore prima di invocare
91      * nuovamente la funzione update().
92      *
93      * Se sono disponibili nuovi dati per la temperatura e l'umidità, la
94      * funzione restituisce true ed è necessario leggere entrambi i valori
95      * il prima possibile, altrimenti non verranno effettuate nuove richieste
96      * di lettura di nuovi dati.
97      *
98      * @return true nuovi dati disponibili per la temperatura e l'umidità
99      * @return false nessun nuovo dato disponibile o si è verificato un errore
100     */
101     bool update();
102
103     /**
104     * @brief Restituisce la temperatura misurata dal sensore SHT20.
105     *
106     * @return float temperatura misurata in gradi Celsius
107     */
108     float get_temperature();
109
110     /**
111     * @brief Restituisce l'umidità misurata dal sensore SHT20.
112     *
113     * @return float umidità misurata in percentuale
114     */
115     float get_humidity();
116
117     /**
118     * @brief Restituisce il codice di errore del sensore SHT20.
119     *
120     * @return int codice di errore
121     */
122     int get_error();
123
124     private:
125     SHT2x sensor; // oggetto della classe SHT2x per la gestione del sensore SHT20
126     uint32_t read_period; // periodo di lettura del sensore SHT20
127     uint32_t read_timer; // timer per le letture del sensore SHT20
128     uint16_t error; // codice di errore del sensore SHT20
129     uint8_t state; // stato del sensore SHT20
130 };
131
132 #endif // SHT20_H

```

Codice 12: File sorgente con le funzioni per la gestione del sensore SHT20

```

1  /**
2  * @file sht20.cpp
3  * @author Giacomo Simonetto
4  * @brief Gestione del sensore SHT20
5  *
6  * Implementazione delle funzioni per la gestione del sensore SHT20.
7  */
8
9  #include "sht20.h"
10

```

```

11 // ----- gestione dello stato del sensore SHT20 -----
12 // -----
13 // -----
14
15 /**
16  * @brief Gestione dello stato del sensore SHT20.
17  *
18  * Il sensore viene trattato come una macchina con i seguenti 6 stati finiti
19  * che si susseguono in sequenza ciclica, con il seguente ordine:
20  *
21  * 0. SHT20_IDLE:
22  *   - il sensore è in pausa e non è in corso alcuna lettura
23  *   - passa a SHT20_READY_FOR_TEMP_REQ dopo 1 secondo dall'ultima lettura
24  *
25  * 1. SHT20_READY_FOR_TEMP_REQ:
26  *   - il sensore è pronto per ricevere la richiesta di lettura della temperatura
27  *   - passa a SHT20_WAIT_TEMP dopo aver inviato la richiesta di lettura della temperatura
28  *
29  * 2. SHT20_WAIT_TEMP:
30  *   - il sensore è in attesa della risposta alla richiesta di lettura della temperatura
31  *   - passa a SHT20_READY_FOR_RH_REQ dopo aver ricevuto la risposta
32  *
33  * 3. SHT20_READY_FOR_RH_REQ:
34  *   - il sensore è pronto per ricevere la richiesta di lettura dell'umidità
35  *   - passa a SHT20_WAIT_RH dopo aver inviato la richiesta di lettura dell'umidità
36  *
37  * 4. SHT20_WAIT_RH:
38  *   - il sensore è in attesa della risposta alla richiesta di lettura dell'umidità
39  *   - passa a SHT20_NEW_DATA dopo aver ricevuto la risposta
40  *
41  * 5. SHT20_NEW_DATA:
42  *   - il sensore ha nuovi dati disponibili per la temperatura e l'umidità
43  *   - passa a SHT20_TEMP_READ o SHT20_RH_READ dopo aver letto uno dei due valori
44  *
45  * 6. SHT20_TEMP_READ:
46  *   - la temperatura è stata letta dal sensore
47  *   - passa a SHT20_IDLE dopo aver letto anche l'umidità
48  *
49  * 7. SHT20_RH_READ:
50  *   - l'umidità è stata letta dal sensore
51  *   - passa a SHT20_IDLE dopo aver letto anche la temperatura
52  */
53
54 // stati del sensore SHT20
55 #define SHT20_IDLE 0 // sensore in pausa
56 #define SHT20_READY_FOR_TEMP_REQ 1 // pronto per richiedere la temperatura
57 #define SHT20_WAIT_TEMP 2 // in attesa di ricevere la temperatura
58 #define SHT20_READY_FOR_RH_REQ 3 // pronto per richiedere l'umidità
59 #define SHT20_WAIT_RH 4 // in attesa di ricevere l'umidità
60 #define SHT20_NEW_DATA 5 // nuovi dati disponibili dal sensore
61 #define SHT20_TEMP_READ 6 // temperatura letta dal sensore
62 #define SHT20_RH_READ 7 // umidità letta dal sensore
63
64 // ----- implementazione della classe sht20 -----
65 // -----
66 // -----
67
68 // costruttore della classe sht20
69 sht20::sht20(uint32_t rp) {
70     sensor = SHT2x(); // inizializzazione oggetto della classe SHT2x
71     state = SHT20_IDLE; // stato del sensore SHT20
72     read_period = rp; // periodo di lettura del sensore SHT20
73     read_timer = OUL; // timer per le letture del sensore SHT20
74     error = OU; // codice di errore del sensore SHT20
75 }
76
77 // inizializzazione del sensore SHT20
78 bool sht20::begin() {
79     // inizializzazione del sensore SHT20 e verifica errori
80     if (!sensor.begin()) {
81         error = sensor.getError();
82         return false;
83     } else {
84         error = 0;
85         return true;
86     }
87 }
88
89 // aggiorna lo stato del sensore SHT20

```

```

90 bool sht20::update() {
91     switch (state) {
92         // --- 0. sensore in attesa
93         case SHT20_IDLE:
94             // verifica se sia trascorso il periodo tra due letture consecutive
95             //if (read_timer == 0UL) {
96             //    read_timer = millis(); // inizializzazione timer
97             //} else
98             if (millis() - read_timer >= read_period) {
99                 read_timer += read_period; // aggiornamento timer richiesta
100                 state = SHT20_READY_FOR_TEMP_REQ; // aggiornamento stato
101             }
102
103             // nessun nuovo dato disponibile
104             return false;
105
106         // --- 1. sensore pronto per inviare la richiesta di lettura della temperatura
107         case SHT20_READY_FOR_TEMP_REQ:
108             // invio richiesta al sensore e verifica errori
109             if (!sensor.requestTemperature()) {
110                 error = sensor.getError(); // recupero codice di errore
111                 return false; // nessun nuovo dato disponibile
112             }
113
114             error = 0; // reset codice di errore
115             state = SHT20_WAIT_TEMP; // aggiornamento stato
116             return false; // nessun nuovo dato disponibile
117
118         // --- 2. sensore in attesa della risposta alla richiesta di lettura della temperatura
119         case SHT20_WAIT_TEMP:
120             // verifica presenza di nuovi dati disponibili per la temperatura
121             // reqTempReady() non lancia errori (controlla solo un timer interno)
122             if (sensor.reqTempReady()) {
123
124                 // recupero temperatura dal sensore e verifica errori
125                 if (!sensor.readTemperature()) {
126                     error = sensor.getError(); // recupero codice di errore
127                     return false; // nessun nuovo dato disponibile
128                 }
129
130                 error = 0; // reset codice di errore
131                 state = SHT20_READY_FOR_RH_REQ; // aggiornamento stato
132             }
133             return false; // nessun nuovo dato disponibile
134
135         // --- 3. sensore pronto per inviare la richiesta di lettura dell'umidità
136         case SHT20_READY_FOR_RH_REQ:
137             // invio richiesta al sensore e verifica errori
138             if (!sensor.requestHumidity()) {
139                 error = sensor.getError(); // recupero codice di errore
140                 return false; // nessun nuovo dato disponibile
141             }
142
143             error = 0; // reset codice di errore
144             state = SHT20_WAIT_RH; // aggiornamento stato
145             return false; // nessun nuovo dato disponibile
146
147         // --- 4. sensore in attesa della risposta alla richiesta di lettura dell'umidità
148         case SHT20_WAIT_RH:
149             // verifica presenza di nuovi dati disponibili per l'umidità
150             // reqHumReady() non lancia errori (controlla solo un timer interno)
151             if (sensor.reqHumReady()) {
152
153                 // recupero umidità dal sensore e verifica errori
154                 if (!sensor.readHumidity()) {
155                     error = sensor.getError(); // recupero codice di errore
156                     return false; // nessun nuovo dato disponibile
157                 }
158
159                 error = 0; // reset codice di errore
160                 state = SHT20_NEW_DATA; // aggiornamento stato
161             }
162             return false; // nessun nuovo dato disponibile
163
164         // --- 5. sensore ha nuovi dati disponibili per la temperatura e l'umidità
165         case SHT20_NEW_DATA:
166             // l'aggiornamento dello stato avviene dopo la lettura dei dati
167             // tramite le funzioni get_temperature() e get_humidity()
168

```

```

169         // nuovi dati disponibili per la temperatura e l'umidità
170         return true;
171
172         // --- x. stato non valido, non si verifica mai
173         default:
174             return false;
175     }
176 }
177
178 // restituisce la temperatura misurata dal sensore SHT20
179 float sht20::get_temperature() {
180     // aggiornamento stato dopo aver letto solo la temperatura
181     if (state == SHT20_NEW_DATA)
182         state = SHT20_TEMP_READ;
183
184     // aggiornamento stato dopo aver letto temperatura e umidità
185     else if (state == SHT20_RH_READ)
186         state = SHT20_IDLE;
187
188     // stato non valido per la lettura della temperatura
189     else if (state != SHT20_TEMP_READ)
190         return NAN;
191
192     // restituisce la temperatura misurata dal sensore SHT20
193     // getTemperature() non lancia errori, fa solo calcoli matematici
194     return sensor.getTemperature();
195 }
196
197 // restituisce l'umidità misurata dal sensore SHT20
198 float sht20::get_humidity() {
199     // aggiornamento stato dopo aver letto solo l'umidità
200     if (state == SHT20_NEW_DATA)
201         state = SHT20_RH_READ;
202
203     // aggiornamento stato dopo aver letto temperatura e umidità
204     else if (state == SHT20_TEMP_READ)
205         state = SHT20_IDLE;
206
207     // stato non valido per la lettura dell'umidità
208     else if (state != SHT20_RH_READ)
209         return NAN;
210
211     // restituisce l'umidità misurata dal sensore SHT20
212     // getHumidity() non lancia errori, fa solo calcoli matematici
213     return sensor.getHumidity();
214 }
215
216 // restituisce il codice di errore del sensore SHT20
217 int sht20::get_error() {
218     return error;
219 }

```

B.7 Files del modulo per la gestione delle comunicazioni seriali

Codice 13: File header con le funzioni per la gestione delle comunicazioni seriali

```
1  /**
2   * @file serial.h
3   * @author Giacomo Simonetto
4   * @brief Gestione della comunicazione seriale dell'incubatrice neonatale
5   *
6   * Dichiarazione delle funzioni per la gestione della comunicazione seriale
7   * tra computer e incubatrice neonatale.
8   */
9
10 #ifndef SERIAL_H
11 #define SERIAL_H
12
13 #include "config.h"
14
15 /**
16  * @brief Stampa il messaggio di avvio su serial monitor.
17  */
18 void serial_boot_message();
19
20 /**
21  * @brief Trasmissione temperatura e umidità misurate.
22  */
23 void serial_datalog();
24
25 /**
26  * @brief Accensione manuale del riscaldatore.
27  */
28 void serial_man_heat_on();
29
30 /**
31  * @brief Spegnimento manuale del riscaldatore.
32  */
33 void serial_man_heat_off();
34
35 /**
36  * @brief Accensione manuale dell'umidificatore.
37  */
38 void serial_man_rh_on();
39
40 /**
41  * @brief Spegnimento manuale dell'umidificatore.
42  */
43 void serial_man_rh_off();
44
45 /**
46  * @brief Accensione manuale dell'illuminazione.
47  */
48 void serial_man_light_on();
49
50 /**
51  * @brief Spegnimento manuale dell'illuminazione.
52  */
53 void serial_man_light_off();
54
55 /**
56  * @brief Accensione automatica del riscaldatore.
57  */
58 void serial_auto_heat_on();
59
60 /**
61  * @brief Spegnimento automatico del riscaldatore.
62  */
63 void serial_auto_heat_off();
64
65 /**
66  * @brief Accensione automatica dell'umidificatore.
67  */
68 void serial_auto_rh_on();
69
70 /**
```

```

71  * @brief Spegnimento automatico dell'umidificatore.
72  */
73  void serial_auto_rh_off();
74
75  /**
76   * @brief Visualizzazione del nuovo setpoint di temperatura.
77   */
78  void serial_new_temp_setpoint();
79
80  /**
81   * @brief Visualizzazione del nuovo setpoint di umidità.
82   */
83  void serial_new_rh_setpoint();
84
85  /**
86   * @brief Stampa il messaggio di refill sul serial monitor.
87   */
88  void serial_refill_message();
89
90  /**
91   * @brief Stampa il messaggio di errore associato al codice di errore
92   * restituito dal sensore SHT20.
93   *
94   * @param errorCode codice di errore
95   */
96  void serial_sht20_error(int errorCode);
97
98  #endif // SERIAL_H

```

Codice 14: File sorgente con le funzioni per la gestione delle comunicazioni seriali

```

1  /**
2   * @file serial.cpp
3   * @author Giacomo Simonetto
4   * @brief Gestione della comunicazione seriale dell'incubatrice neonatale
5   *
6   * Implementazione delle funzioni per la gestione della comunicazione seriale
7   * tra computer e incubatrice neonatale.
8   */
9
10 #include "serial.h"
11 #include "utils.h"
12
13 // messaggio di avvio su serial monitor
14 void serial_boot_message() {
15     Serial.println(F("BOOT: avvio incubatrice neonatale"));
16 }
17
18 // trasmissione temperatura, umidità e stato del sistema
19 void serial_datalog() {
20     Serial.print(F("LOG: T: "));
21     Serial.print(data_to_string(status.temp_sht20));
22     Serial.print(F("°C | RH: "));
23     Serial.print(data_to_string(status.rh_sht20));
24     Serial.print(F("% | Set_T: "));
25     Serial.print(data_to_string(status.temp_setpoint));
26     Serial.print(F("°C | Set_RH: "));
27     Serial.print(data_to_string(status.rh_setpoint));
28     Serial.print(F("% | Ctrl: "));
29     Serial.print(status.manual_ctrl ? F("MAN") : F("AUTO"));
30     Serial.print(F(" | Error_Code: "));
31     Serial.print(status.error_code);
32     #if TEMP_CTRL == 2
33         Serial.print(F(" | T_PID_p: "));
34         Serial.print(temp_pid.proportional);
35         Serial.print(F(" | T_PID_i: "));
36         Serial.print(temp_pid.integral);
37         Serial.print(F(" | T_PID_d: "));
38         Serial.print(temp_pid.derivative);
39         Serial.print(F(" | T_PID_out: "));
40         Serial.print(temp_pid.output);
41     #endif
42     Serial.print(F(" | T_PWM_norm: "));
43     Serial.print(map(status.temp_pwm_value, 0, PWM_PERIOD, PID_MIN_OUTPUT, PID_MAX_OUTPUT));
44     #if RH_CTRL == 2

```

```

45     Serial.print(F(" | RH_PID_p: "));
46     Serial.print(rh_pid.proportional);
47     Serial.print(F(" | RH_PID_i: "));
48     Serial.print(rh_pid.integral);
49     Serial.print(F(" | RH_PID_d: "));
50     Serial.print(rh_pid.derivative);
51     Serial.print(F(" | RH_PID_out: "));
52     Serial.print(rh_pid.output);
53     #endif
54     Serial.print(F(" | RH_PWM_norm: "));
55     Serial.println(map(status.rh_pwm_value, 0, PWM_PERIOD, PID_MIN_OUTPUT, PID_MAX_OUTPUT));
56 }
57
58 // visualizzazione del nuovo setpoint di temperatura
59 void serial_new_temp_setpoint() {
60     Serial.print(F("SET0: New Set_T: "));
61     Serial.print(data_to_string(status.temp_setpoint));
62     Serial.println(F("°C"));
63 }
64
65 // visualizzazione del nuovo setpoint di umidità
66 void serial_new_rh_setpoint() {
67     Serial.print(F("SET1: New Set_RH: "));
68     Serial.print(data_to_string(status.rh_setpoint));
69     Serial.println(F('%'));
70 }
71
72 // accensione manuale del riscaldatore
73 void serial_man_heat_on() {
74     Serial.println(F("ACT0: Manual Heat On"));
75 }
76
77 // spegnimento manuale del riscaldatore
78 void serial_man_heat_off() {
79     Serial.println(F("ACT1: Manual Heat Off"));
80 }
81
82 // accensione manuale dell'umidificatore
83 void serial_man_rh_on() {
84     Serial.println(F("ACT2: Manual RH On"));
85 }
86
87 // spegnimento manuale dell'umidificatore
88 void serial_man_rh_off() {
89     Serial.println(F("ACT3: Manual RH Off"));
90 }
91
92 // accensione manuale dell'illuminazione
93 void serial_man_light_on() {
94     Serial.println(F("ACT4: Manual Light On"));
95 }
96
97 // spegnimento manuale dell'illuminazione
98 void serial_man_light_off() {
99     Serial.println(F("ACT5: Manual Light Off"));
100 }
101
102 // accensione automatica del riscaldatore
103 void serial_auto_heat_on() {
104     Serial.println(F("ACT6: Automatic Heat On"));
105 }
106
107 // spegnimento automatico del riscaldatore
108 void serial_auto_heat_off() {
109     Serial.println(F("ACT7: Automatic Heat Off"));
110 }
111
112 // accensione automatica dell'umidificatore
113 void serial_auto_rh_on() {
114     Serial.println(F("ACT8: Automatic RH On"));
115 }
116
117 // spegnimento automatico dell'umidificatore
118 void serial_auto_rh_off() {
119     Serial.println(F("ACT9: Automatic RH Off"));
120 }
121
122 // stampa il messaggio di refill sul serial monitor
123 void serial_refill_message() {

```

```

124 Serial.println(F("ERR-: Refill water in the humidifier"));
125 Serial.println(F("ACT9: Automatic RH Off"));
126 }
127
128 // stampa il messaggio di errore del sensore SHT20
129 void serial_sht20_error(int errorCode) {
130     switch (errorCode) {
131         case SHT2x_ERR_WRITECMD:
132             Serial.println(F("ERR-: SHT20 - Error during I2C write command"));
133             break;
134         case SHT2x_ERR_READBYTES:
135             Serial.println(F("ERR-: SHT20 - Error during I2C read bytes"));
136             break;
137         case SHT2x_ERR_HEATER_OFF:
138             Serial.println(F("ERR-: SHT20 - Failed to switch off the internal heater"));
139             break;
140         case SHT2x_ERR_NOT_CONNECT:
141             Serial.println(F("ERR-: SHT20 - Sensor not connected or does not acknowledge on the I2C bus"));
142             break;
143         case SHT2x_ERR_CRC_TEMP:
144             Serial.println(F("ERR-: SHT20 - CRC check failed for the temperature reading"));
145             break;
146         case SHT2x_ERR_CRC_HUM:
147             Serial.println(F("ERR-: SHT20 - CRC check failed for the humidity reading"));
148             break;
149         case SHT2x_ERR_CRC_STATUS:
150             Serial.println(F("ERR-: SHT20 - CRC check failed for the status register"));
151             break;
152         case SHT2x_ERR_HEATER_COOLDOWN:
153             Serial.println(F("ERR-: SHT20 - Heater is in its mandatory cool-down period and cannot be
154                               ↪ re-enabled yet"));
155             break;
156         case SHT2x_ERR_HEATER_ON:
157             Serial.println(F("ERR-: SHT20 - Failed to switch on the internal heater"));
158             break;
159         case SHT2x_ERR_RESOLUTION:
160             Serial.println(F("ERR-: SHT20 - Invalid resolution parameter provided"));
161             break;
162         default:
163             Serial.println(F("ERR-: SHT20 - Unknown error occurred"));
164             break;
165     }
166 }

```

B.8 Files del modulo per la gestione del controllore PID

Codice 15: File header con le funzioni per la gestione del controllore PID

```
1  /**
2   * @file pid.h
3   * @author Giacomo Simonetto
4   * @brief Implementazione dei controllori PID.
5   *
6   * Dichiarazione delle funzioni per la gestione dei controllori PID per la
7   * temperatura e l'umidità dell'incubatrice neonatale.
8   */
9
10 #ifndef PID_H
11 #define PID_H
12
13 #include "config.h"
14
15 /**
16  * @brief Gestione del controllore PID
17  *
18  * L'output del pid viene aggiornato ogni volta che inizia un nuovo ciclo del
19  * segnale PWM, quindi ogni CTRL_PWM_PERIOD millisecondi. Le misurazioni che
20  * avvengono tra un aggiornamento dell'output e il successivo vengono salvate
21  * all'interno di accumulatori interni per calcolare successivamente le tre
22  * componenti del PID (proporzionale, integrale e derivativa).
23  *
24  * Si avrà quindi una funzione invocata all'acquisizione dei nuovi dati per
25  * fornire le nuove misurazioni da utilizzare nel futuro calcolo dell'output
26  * del PID e una funzione invocata prima dell'inizio di ogni ciclo del PWM
27  * per aggiornare l'output del controllore in base ai dati accumulati fino
28  * a quel momento.
29  *
30  * Per evitare che vecchi dati accumulati prima della disattivazione del
31  * controllo automatico influenzino il calcolo dell'output del PID alla
32  * riattivazione del controllo automatico, è necessario resettare gli
33  * accumulatori interni del PID tramite la funzione apposita.
34  */
35
36
37 /**
38  * @brief Inizializza i parametri del PID.
39  *
40  * Salva i parametri del PID all'interno della struct, inizializzando le
41  * varie variabili interne del PID.
42  *
43  * Il parametro anti_windup è opzionale, se non specificato viene impostato
44  * a 1 (clamping).
45  *
46  * @param pid reference alla struct del PID da inizializzare
47  * @param kp const reference al guadagno proporzionale
48  * @param ki const reference al guadagno integra
49  * @param kd const reference al guadagno derivativo
50  * @param kw const reference al guadagno della back calculation (anti-windup)
51  * @param anti_windup const reference all'opzione di anti-windup da utilizzare
52  */
53 void pid_init(struct pid &pid, const float &kp, const float &ki, const float &kd, const float &kw, const
↳ uint8_t &anti_windup = 1);
54
55 /**
56  * @brief Salva una nuova misurazione negli accumulatori del PID.
57  *
58  * Salva l'errore e la misurazione in apposite variabili interne del PID
59  * che verranno utilizzate per calcolare l'output del PID alla successiva
60  * invocazione della funzione pid_update_output().
61  *
62  * Si assume che i dati forniti alla funzione vengano acquisiti dai sensori
63  * ad intervalli di tempo costanti ogni PID_DATA_PERIOD millisecondi.
64  * All'interno della funzione non si effettua nessun controllo temporale per
65  * cui il rispetto di tale condizione è responsabilità del chiamante.
66  *
67  * Non ci sono vincoli sul tempo che intercorre da quando i dati vengono
68  * acquisiti dal sensore e quando vengono passati alla funzione.
69  *
70  * @param pid reference alla struct del PID da aggiornare
71  * @param error const reference all'errore tra setpoint e valore misurato
72  * @param measure const reference al valore misurato dal sensore
```

```

73  */
74  void pid_add_data(struct pid &pid, const int16_t &error, const int16_t &measure);
75
76  /**
77   * @brief Aggiorna l'output del controllore PID.
78   *
79   * Calcola il nuovo output del PID in base ai dati salvati negli accumulatori
80   * interni dalla funzione pid_add_data() nel tempo intercorso dall'ultimo
81   * aggiornamento dell'output del PID.
82   *
83   * Si prevede che l'output del PID venga aggiornato ad intervalli di tempo
84   * costanti ogni PID_UPDATE_PERIOD millisecondi. All'interno della funzione
85   * non si effettua nessun controllo temporale per cui il rispetto di tale
86   * condizione è responsabilità del chiamante.
87   *
88   * @param pid reference alla struct del PID da aggiornare
89   * @return float output del PID limitato tra CTRL_MIN_OUTPUT e CTRL_MAX_OUTPUT
90   */
91  float pid_update_output(struct pid &pid);
92
93  /**
94   * @brief Resetta gli accumulatori del PID.
95   *
96   * Resetta gli accumulatori interni del pid che contengono i dati delle
97   * misurazioni ricevute dall'ultimo aggiornamento dell'output del PID.
98   *
99   * Deve essere invocata ogni volta che si riattiva il controllo automatico
100  * del PID dopo un periodo di controllo manuale, per evitare che i vecchi
101  * dati memorizzati influenzino il successivo calcolo dell'output del PID.
102  *
103  * @param pid reference alla struct del PID da resettare
104  */
105  void pid_reset_accumulators(struct pid &pid);
106
107  #endif // PID_H

```

Codice 16: File sorgente con le funzioni per la gestione del controllore PID

```

1  /**
2   * @file pid.cpp
3   * @author Giacomo Simonetto
4   * @brief Implementazione dei controllori PID.
5   *
6   * Implementazione delle funzioni per la gestione dei controllori PID per la
7   * temperatura e l'umidità dell'incubatrice neonatale.
8   */
9
10 #include "pid.h"
11
12 // inizializza i parametri per i controllori PID
13 void pid_init(struct pid &pid, const float &kp, const float &ki, const float &kd, const float &kw, const
↳ uint8_t &anti_windup) {
14     // inizializzazione dei guadagni del PID
15     pid.kp = kp;
16     pid.ki = ki;
17     pid.kd = kd;
18     pid.kw = kw;
19
20     // inizializzazione delle componenti del PID
21     pid.proportional = 0.0f;
22     pid.integral = 0.0f;
23     pid.derivative = 0.0f;
24     pid.output = 0.0f;
25
26     // inizializzazione del numero di misurazioni
27     pid.data_count = 0;
28     pid.expected_data_count = PID_UPDATE_PERIOD / PID_DATA_PERIOD;
29
30     // inizializzazione delle variabili e accumulatori intermedi per calcolo PID
31     pid.deriv_c1 = -6.0f / pid.expected_data_count / (pid.expected_data_count + 1) / (PID_DATA_PERIOD /
↳ 1000.0f);
32     pid.deriv_c2 = 12.0f / pid.expected_data_count / (pid.expected_data_count * pid.expected_data_count -
↳ 1) / (PID_DATA_PERIOD / 1000.0f);
33     pid.sum1 = 0L;
34     pid.sum2 = 0L;

```

```

35 pid.sum3 = 0L;
36
37 // inizializzazione del metodo di anti-windup da utilizzare
38 pid.anti_windup = anti_windup;
39 }
40
41
42 // salva una nuova misurazione negli accumulatori del PID
43 void pid_add_data(struct pid &pid, const int16_t &error, const int16_t &measure) {
44
45     /**
46      * NOTE IMPLEMENTATIVE:
47      *
48      * Le grandezze richieste per aggiornare l'output sono:
49      * - sum1:
50      *   somma degli errori tra setpoint e valore misurato, utilizzato per
51      *   la componente proporzionale, integrale e derivativa del PID
52      *
53      * - sum2:
54      *   somma dei valori misurati, utilizzato per la componente derivativa
55      *   del PID
56      *
57      * - sum3:
58      *   somma dei prodotti tra indice della misurazione e valore misurato,
59      *   utilizzato per la componente derivativa del PID
60      *
61      * - data_count:
62      *   numero di misurazioni ricevute ed elaborate, utilizzato per il calcolo
63      *   delle componenti proporzionale, integrale e derivativa del PID
64      */
65
66     pid.sum1 += error;
67     pid.sum2 += measure;
68     pid.sum3 += (int32_t)pid.data_count * (int32_t)measure;
69     pid.data_count++;
70 }
71
72 // aggiorna l'output del controllore PID
73 float pid_update_output(struct pid &pid) {
74
75     /**
76      * NOTE IMPLEMENTATIVE:
77      *
78      * L'output "puro" è calcolato dalla successiva formula:
79      *
80      * pid.output = pid.proportional + pid.integral + pid.derivative
81      *
82      * Per calcolare l'output del PID, si effettuano i seguenti passaggi:
83      * 1. verifica assenza di dati da elaborare
84      * 2. calcolo della componente proporzionale
85      * 3. calcolo della componente integrale
86      * 4. calcolo della componente derivativa
87      * 5. anti-windup della componente integrale
88      * 6. limitazioni dell'output
89      *
90      * ----- 1. verifica dell'assenza di dati da elaborare -----
91      *
92      * In caso di problemi come mancanza totale di misurazioni, l'output del PID
93      * viene impostato al valore minimo, in modo da spegnere l'attuatore e non
94      * far peggiorare la situazione.
95      *
96      * ----- 2. calcolo della componente proporzionale -----
97      *
98      * La componente proporzionale è calcolata come il prodotto tra il guadagno
99      * proporzionale kp e l'errore medio del ciclo di aggiornamento del PID.
100     *
101     * Se ci dovessero essere problemi come mancanza di misurazioni, l'errore
102     * medio calcolato con i dati disponibili è comunque una buona stima di
103     * quello reale e il disturbo introdotto scompare totalmente al successivo
104     * aggiornamento dell'output del PID.
105     *
106     * mean_error = sum_error / data_count
107     * pid.proportional = kp * mean_error
108     *
109     *
110     * ----- 3. calcolo della componente integrale -----
111     *
112     * La componente integrale è calcolata come il prodotto tra il guadagno

```

```

114 * integrale ki, l'errore accumulato nel ciclo di aggiornamento del PID e
115 * l'intervallo tra due misurazioni consecutive.
116 *
117 * Se ci dovessero essere problemi come mancanza di misurazioni, i valori
118 * mancanti vengono sostituiti con l'errore medio calcolato per la parte
119 * proporzionale, essendo una buona stima. Il disturbo introdotto verrà
120 * propagato anche nei cicli successivi, ma verrà corretto grazie alla
121 * presenza del feedback del controllore PID.
122 *
123 * pid.integral = ki * mean_error * expected_data_count * (PID_DATA_PERIOD / 1000.0f)
124 *
125 *
126 * ----- 4. calcolo della componente derivativa -----
127 *
128 * La componente derivativa è calcolata come il prodotto tra il guadagno
129 * derivativo kd e la derivata della grandezza da controllare, calcolata
130 * tramite la formula ottenuta applicando il metodo dei minimi quadrati
131 * alla regressione lineare sulle misurazioni passate.
132 *
133 * Si calcola la derivata della grandezza da controllare e non dell'errore
134 * tra setpoint e valore misurato in modo da evitare i "derivative kick"
135 * che si verificano al cambiamento del setpoint.
136 *
137 * pid.derivative = kd * ( -deriv_c1 * sum2 + deriv_c2 * sum3);
138 *
139 *
140 * 
$$\frac{dy}{dt} = - \frac{6}{Ts * N * (N + 1)} * \sum_{i=0}^{N-1} y_i + \frac{12}{Ts * N * (N^2 - 1)} * \sum_{i=0}^{N-1} (i * y_i)$$

141 *
142 * 
$$\frac{\text{deriv\_c1}}{\text{sum2}} \quad \frac{\text{deriv\_c2}}{\text{sum3}}$$

143 *
144 * con:
145 * - N = numero di dati passati
146 * - Ts = intervallo tra due misurazioni consecutive
147 * - y_i = valore misurato i-esimo istante
148 * - i = indice del dato passato (0 = più vecchio, N-1 = più recente)
149 *
150 * Siccome i due coefficienti coeff1 e coeff2 sono costanti, vengono
151 * precalcolati in fase di inizializzazione e memorizzati in due apposite
152 * costanti deriv_c1 e deriv_c2.
153 *
154 * Se ci dovessero essere problemi come mancanza di misurazioni, vengono
155 * utilizzati solo i valori disponibili siccome è parecchio complesso
156 * interpolare e integrare i valori mancanti all'interno delle sommatorie.
157 * Inoltre il disturbo introdotto scompare totalmente all'aggiornamento
158 * successivo dell'output e la regressione rimane sempre una buona stima
159 * della derivata reale. L'unico svantaggio è che bisogna ricalcolare le
160 * costanti deriv_c1 e deriv_c2 in funzione del numero di dati disponibili.
161 * Nel caso in cui si ha un solo dato, la componente derivativa viene
162 * impostata a zero.
163 *
164 *
165 * ----- 5. anti-windup della componente integrale -----
166 *
167 * Si implementano tre opzioni per la gestione del windup della componente
168 * integrale del PID (no windup, clamping e back calculation) selezionabili
169 * tramite la flag anti_windup della struct del PID:
170 *
171 * 0. no anti-windup:
172 * la componente integrale viene sempre sommata all'output del PID
173 *
174 * 1. clamping:
175 * se l'output del PID è superiore al limite massimo e l'errore è
176 * positivo, o viceversa se l'output del PID è inferiore al limite
177 * minimo e l'errore è negativo, il nuovo contributo della componente
178 * integrale viene scartato sia per il calcolo dell'output del PID che
179 * per l'accumulo dell'errore integrale
180 *
181 * 2. back calculation:
182 * la componente integrale viene compensata da un termine proporzionale
183 * alla differenza tra l'output puro del PID e l'output limitato in modo
184 * da far riportare l'output del PID all'interno dei limiti
185 *
186 *
187 * ----- 6. limitazioni dell'output -----
188 *
189 * Prima di essere restituito al chiamante, l'output del PID viene limitato
190 * tra le due costanti PID_MIN_OUTPUT e PID_MAX_OUTPUT.
191 */
192

```

```

193 // 1. verifica dell'assenza di dati da elaborare
194 if (pid.data_count == 0) {
195     pid.sum1 = 0L;
196     pid.sum2 = 0UL;
197     pid.sum3 = 0UL;
198     return PID_MIN_OUTPUT;
199 }
200
201 // 2. calcolo della componente proporzionale
202 float mean_error = (float)pid.sum1 / pid.data_count;
203 pid.proportional = pid.kp * mean_error;
204
205 // 3. calcolo del contributo della componente integrale
206 float integral_contrib = pid.ki * mean_error * pid.expected_data_count * (PID_DATA_PERIOD / 1000.0f);
207
208 // 4. calcolo della componente derivativa
209 if (pid.data_count == pid.expected_data_count) {
210     pid.derivative = pid.kd * (pid.deriv_c1 * pid.sum2 + pid.deriv_c2 * pid.sum3);
211 } else if (pid.data_count == 1) {
212     pid.derivative = 0.0f;
213 } else {
214     float c1 = -6.0f / pid.data_count / (pid.data_count + 1.0f) / (PID_DATA_PERIOD / 1000.0f);
215     float c2 = 12.0f / pid.data_count / (pid.data_count * pid.data_count - 1.0f) / (PID_DATA_PERIOD /
        ↪ 1000.0f);
216     pid.derivative = pid.kd * (c1 * pid.sum2 + c2 * pid.sum3);
217 }
218
219 // calcolo dell'output temporaneo del PID
220 pid.output = pid.proportional + pid.integral + integral_contrib - pid.derivative;
221
222 // 5. anti-windup della componente integrale del PID
223 switch (pid.anti_windup) {
224     case 0: // no anti-windup
225         pid.integral += integral_contrib;
226         break;
227
228     case 1: // clamping
229         if (!(pid.output > PID_MAX_OUTPUT && mean_error > 0.0f) && !(pid.output < PID_MIN_OUTPUT &&
        ↪ mean_error < 0.0f))
230             pid.integral += integral_contrib;
231         break;
232
233     case 2: // back calculation
234         pid.integral += integral_contrib + pid.kw * (constrain(pid.output, PID_MIN_OUTPUT, PID_MAX_OUTPUT)
        ↪ - pid.output) * (PID_UPDATE_PERIOD / 1000.0f);
235         break;
236 }
237
238 // calcolo dell'output finale del PID
239 pid.output = pid.proportional + pid.integral - pid.derivative;
240
241 // azzeramento delle variabili per il prossimo ciclo del PID
242 pid.sum1 = 0L;
243 pid.sum2 = 0UL;
244 pid.sum3 = 0UL;
245 pid.data_count = 0;
246
247 // 6. limitazione dell'output del PID tra PID_MIN_OUTPUT e PID_MAX_OUTPUT
248 return constrain(pid.output, PID_MIN_OUTPUT, PID_MAX_OUTPUT);
249 }
250
251 // azzeramento degli accumulatori del PID
252 void pid_reset_accumulators(struct pid &pid) {
253     pid.sum1 = 0L;
254     pid.sum2 = 0L;
255     pid.sum3 = 0L;
256     pid.data_count = 0;
257 }

```

B.9 File principale del firmware

Codice 17: File principale del firmware

```
1  /**
2   * @file main.cpp
3   * @author Giacomo Simonetto
4   * @brief Firmware per incubatrice neonatale
5   *
6   * Firmware per il controllo di un'incubatrice neonatale, con gestione di
7   * temperatura, umidità e pannello frontale con display lcd e switch.
8   */
9
10
11 // -----
12 // ----- dipendenze -----
13 // -----
14
15 #include "config.h" // parametri di configurazione e variabili globali
16 #include "error.h" // gestione errori e messaggi di errore
17 #include "utils.h" // funzioni di utilità generiche
18 #include "pid.h" // gestione controllori PID
19 #include "actuators.h" // gestione accensione e spegnimento degli attuatori
20 #include "serial.h" // gestione messaggi su serial monitor
21 #include "lcd.h" // gestione messaggi su display lcd
22
23
24 // -----
25 // ----- variabili globali temporanee -----
26 // -----
27
28 int16_t sensor_read = 0; // valore appena letto dal sensore SHT20
29 int16_t pot_read = 0; // valore appena letto dai potenziometri
30 int16_t new_temp_setpoint = 0; // setpoint appena calcolato
31 int16_t new_rh_setpoint = 0; // setpoint appena calcolato
32
33
34 // -----
35 // ----- helper functions per il main -----
36 // -----
37
38 /**
39  * @brief Attivazione del controllo manuale degli attuatori.
40  *
41  * Imposta lo stato degli attuatori in modo che coincida con lo stato degli
42  * switch del pannello frontale e disabilita il controllo automatico settando
43  * opportunamente il flag status.manual_ctrl a MANUAL_CTRL.
44  */
45 void set_manual_ctrl();
46
47 /**
48  * @brief Attivazione del controllo automatico degli attuatori.
49  *
50  * Riattiva il controllo automatico degli attuatori resettando i timer del
51  * ciclo PWM, resettando i vecchi dati accumulati dai PID e impostando il
52  * flag status.manual_ctrl a AUTO_CTRL.
53  */
54 void set_auto_ctrl();
55
56 /**
57  * @brief Verifica se è necessario un refill dell'acqua.
58  *
59  * Questa funzione verifica se è necessario effettuare un refill dell'acqua
60  * dell'incubatrice. Controlla se il periodo di tempo in cui l'umidificatore
61  * è stato acceso supera il valore di REFILL_INTERVAL. Se è necessario un
62  * refill, la funzione spegne l'umidificatore e accende il led di refill,
63  * attivando il blocco automatico dell'umidificatore.
64  *
65  * Per far ripristinare il funzionamento dell'umidificatore, è necessario
66  * effettuare un refill dell'acqua e successivamente riavviare l'incubatrice
67  * neonatale dal tasto reset della scheda Arduino. In questo modo tutti i
68  * contatori vengono azzerati.
69  *
70  * Per funzionare correttamente, questa funzione deve essere invocata ad ogni
71  * iterazione del loop principale.
72  *
73  * @return true refill necessario, umidificatore spento e led di refill acceso
```

```

74  * @return false refill non necessario, nessuna azione eseguita
75  */
76  bool check_refill();
77
78
79  // -----
80  // ----- setup function -----
81  // -----
82
83  void setup() {
84      // inizializzazione seriale e I2C
85      Serial.begin(115200);
86
87      // inizializzazione protocollo Wire per I2C
88      Wire.begin();
89
90      // inizializzazione display lcd e stampa messaggio di avvio
91      lcd.init();
92      lcd.clear();
93      lcd.backlight();
94
95      // messaggio di avvio su serial monitor e display lcd
96      serial_boot_message();
97      lcd_boot_message();
98
99      // inizializzazione pin
100     pinMode(ALARM_LED, OUTPUT);
101     pinMode(HEAT_LED, OUTPUT);
102     pinMode(RH_LED, OUTPUT);
103     pinMode(REFILL_LED, OUTPUT);
104
105     pinMode(HEAT_RELAY, OUTPUT);
106     pinMode(RH_RELAY, OUTPUT);
107     pinMode(FAN_RELAY, OUTPUT);
108     pinMode(LIGHT_RELAY, OUTPUT);
109
110     pinMode(HEAT_SW, INPUT);
111     pinMode(RH_SW, INPUT);
112     pinMode(LIGHT_SW, INPUT);
113
114     // inizializzazione stato iniziale dei pin con led, relè
115     digitalWrite(ALARM_LED, LED_OFF);
116     digitalWrite(HEAT_LED, LED_OFF);
117     digitalWrite(RH_LED, LED_OFF);
118     digitalWrite(REFILL_LED, LED_OFF);
119
120     digitalWrite(HEAT_RELAY, RELAY_OFF);
121     digitalWrite(RH_RELAY, RELAY_OFF);
122     digitalWrite(FAN_RELAY, RELAY_OFF);
123     digitalWrite(LIGHT_RELAY, RELAY_OFF);
124
125     // inizializzazione stato iniziale delle variabili globali
126     memset(&status, 0, sizeof(status));
127     memset(&timers, 0, sizeof(timers));
128     memset(&temp_pid, 0, sizeof(temp_pid));
129     memset(&rh_pid, 0, sizeof(rh_pid));
130
131     // inizializzazione setpoint di temperatura e umidità
132     status.temp_setpoint = new_temp_setpoint = TEMP_DEF_SP;
133     status.rh_setpoint = new_rh_setpoint = RH_DEF_SP;
134
135     // inizializzazione controllori PID
136     #if TEMP_CTRL == 2
137         pid_init(temp_pid, TEMP_PID_KP, TEMP_PID_KI, TEMP_PID_KD, TEMP_PID_KW, TEMP_PID_WINDUP);
138     #endif
139     #if RH_CTRL == 2
140         pid_init(rh_pid, RH_PID_KP, RH_PID_KI, RH_PID_KD, RH_PID_KW, RH_PID_WINDUP);
141     #endif
142
143     // inizializzazione sensore SHT20 e controllo errori del sensore
144     while (!sensor.begin()) {
145         lcd_sht20_error(sensor.get_error()); // stampa messaggio su display lcd
146         serial_sht20_error(sensor.get_error()); // stampa messaggio su serial monitor
147         digitalWrite(ALARM_LED, LED_ON); // accensione led di allarme
148         delay(1000); // attesa di 1 secondo prima di riprovare
149     }
150     digitalWrite(ALARM_LED, LED_OFF); // spegnimento led in assenza di errori
151
152     // ritardo prima di iniziare il loop principale

```

```

153 delay(2000);
154
155 // inizializzazione valori potenziometri e switch
156 status.temp_pot_value = analogRead(TEMP_POT);
157 status.rh_pot_value = analogRead(RH_POT);
158 status.heat_sw = digitalRead(HEAT_SW);
159 status.rh_sw = digitalRead(RH_SW);
160 status.light_sw = digitalRead(LIGHT_SW);
161
162 // attivazione luce in base al valore dello switch
163 if (status.light_sw)
164     light_turn_on(); // accensione illuminazione
165
166 // inizializzazione timers del sensore e del ciclo pwm
167 timers.start_pwm = millis();
168 sensor.init_timer();
169 }
170
171
172 // -----
173 // ----- loop function -----
174 // -----
175
176 void loop() {
177     // -----
178     // ----- gestione switch riscaldatore, umidificatore e illuminazione -----
179     // -----
180
181     // ----- cambio stato switch riscaldatore -----
182     if (millis() - timers.heat_sw >= SW_READ_PERIOD && status.heat_sw != digitalRead(HEAT_SW)) {
183         timers.heat_sw = millis(); // aggiornamento timer
184         status.heat_sw = !status.heat_sw; // aggiornamento stato switch
185
186         // switch riscaldatore attivo
187         if (status.heat_sw) {
188             heat_turn_on(); // accensione riscaldatore e led associato
189             lcd_man_heat_on(); // stampa messaggio di accensione sul display
190             serial_man_heat_on(); // stampa messaggio di accensione su serial monitor
191         }
192
193         // switch riscaldatore spento
194         else {
195             heat_turn_off(); // spegnimento riscaldatore e led associato
196             lcd_man_heat_off(); // stampa messaggio di spegnimento sul display
197             serial_man_heat_off(); // stampa messaggio di spegnimento su serial monitor
198         }
199
200         // attivazione controllo manuale
201         set_manual_ctrl();
202     }
203
204     // ----- cambio stato switch umidificatore -----
205     if (millis() - timers.rh_sw >= SW_READ_PERIOD && status.rh_sw != digitalRead(RH_SW)) {
206         timers.rh_sw = millis(); // aggiornamento timer
207         status.rh_sw = !status.rh_sw; // aggiornamento stato switch
208
209         // avviso di refill acqua
210         if (status.refill_led && status.rh_sw) {
211             lcd_refill_message(); // stampa messaggio refill sul display
212         }
213
214         // switch umidificatore attivo
215         else if (!status.refill_led && status.rh_sw) {
216             rh_turn_on(); // accensione umidificatore e led associato
217             lcd_man_rh_on(); // stampa messaggio di accensione sul display
218             serial_man_rh_on(); // stampa messaggio di accensione su serial monitor
219         }
220
221         // switch umidificatore spento
222         else if (!status.refill_led && !status.rh_sw) {
223             rh_turn_off(); // spegnimento umidificatore e led associato
224             lcd_man_rh_off(); // stampa messaggio di spegnimento sul display
225             serial_man_rh_off(); // stampa messaggio di spegnimento su serial monitor
226         }
227
228         // attivazione controllo manuale
229         set_manual_ctrl();
230     }
231 }

```

```

232 // ----- cambio stato switch illuminazione -----
233 if (millis() - timers.light_sw >= SW_READ_PERIOD && status.light_sw != digitalRead(LIGHT_SW)) {
234     timers.light_sw = millis(); // aggiornamento timer
235     status.light_sw = !status.light_sw; // aggiornamento stato switch
236
237     // switch illuminazione attivo
238     if (status.light_sw) {
239         light_turn_on(); // accensione illuminazione
240         lcd_man_light_on(); // stampa messaggio di accensione sul display
241         serial_man_light_on(); // stampa messaggio di accensione su serial monitor
242     }
243     // switch illuminazione spento
244     else {
245         light_turn_off(); // spegnimento illuminazione
246         lcd_man_light_off(); // stampa messaggio di spegnimento sul display
247         serial_man_light_off(); // stampa messaggio di spegnimento su serial monitor
248     }
249 }
250
251
252 // -----
253 // ----- aggiornamento setpoint di temperatura e umidità -----
254 // -----
255
256 /**
257  * @brief Aggiornamento dei setpoint di temperatura e umidità.
258  *
259  * Il processo di aggiornamento del setpoint avviene nelle seguenti fasi:
260  *
261  * 1. lettura del potenziometro ogni XX_POT_TIMER millisecondi
262  * 2. controllo di soglia per eliminare disturbi e rumore elettrico
263  * 3. calcolo del nuovo setpoint di temperatura o umidità e verifica che
264  *    rientri nei limiti previsti XX_MIN_SP e XX_MAX_SP, con conseguente
265  *    stampa del nuovo valore sul display lcd
266  * 4. avvio del timer di ultima modifica del setpoint se il nuovo valore
267  *    è diverso dal precedente e attivazione del controllo automatico
268  * 5. aggiornamento del setpoint se il nuovo valore è diverso dal precedente
269  *    e se è trascorso il tempo di ritardo previsto SETPOINT_UPDATE_DELAY
270  *    e stampa del nuovo setpoint su serial monitor
271  *
272  * Nel caso di potenziometri ad alta impedenza, è possibile che la lettura
273  * analogica del valore del potenziometro venga influenzata dalla carica
274  * residua del condensatore dell'ADC, provocando valori errati. Questo
275  * fenomeno è denominato crosstalk dell'ADC. Per evitarlo, viene effettuata
276  * una ulteriore lettura preliminare in modo che il condensatore abbia tempo
277  * di caricarsi correttamente prima della lettura effettiva.
278  */
279
280 // controllo timer per lettura potenziometro temperatura
281 if (millis() - timers.temp_pot >= TEMP_POT_READ_PERIOD) {
282     timers.temp_pot += TEMP_POT_READ_PERIOD; // aggiornamento timer
283     analogRead(TEMP_POT); // lettura per evitare crosstalk dell'ADC
284     pot_read = analogRead(TEMP_POT); // 1. lettura potenziometro temperatura
285
286     // 2. controllo threshold
287     if (abs(status.temp_pot_value - pot_read) >= TEMP_POT_THLD) {
288         status.temp_pot_value = pot_read; // aggiornamento valore potenziometro temperatura
289
290         // 3. calcolo nuovo setpoint di temperatura
291         new_temp_setpoint = map(pot_read, TEMP_POT_MIN, TEMP_POT_MAX, TEMP_MIN_SP / TEMP_STEP_SP,
292                                TEMP_MAX_SP / TEMP_STEP_SP) * TEMP_STEP_SP;
293         if (new_temp_setpoint < TEMP_MIN_SP) new_temp_setpoint = TEMP_MIN_SP; // controllo limite
294         ↪ inferiore
295         if (new_temp_setpoint > TEMP_MAX_SP) new_temp_setpoint = TEMP_MAX_SP; // controllo limite
296         ↪ superiore
297
298         // stampa su display lcd
299         lcd_new_temp_setpoint(new_temp_setpoint);
300
301         // 4. controllo variazioni del setpoint di temperatura
302         if (status.temp_setpoint != new_temp_setpoint) {
303             timers.temp_setpoint = millis(); // aggiornamento timer
304             set_auto_ctrl(); // attivazione controllo automatico
305         }
306     }
307 }
308
309 // 5. aggiornamento effettivo del setpoint di temperatura
310 if (status.temp_setpoint != new_temp_setpoint && millis() - timers.temp_setpoint >= SP_UPDATE_DELAY) {

```

```

308     status.temp_setpoint = new_temp_setpoint; // aggiornamento setpoint precedente
309     serial_new_temp_setpoint(); // stampa su serial monitor
310
311 }
312
313 // controllo timer per lettura potenziometro umidità
314 if (millis() - timers.rh_pot >= RH_POT_READ_PERIOD) {
315     analogRead(RH_POT); // lettura per evitare crosstalk dell'ADC
316     timers.rh_pot += RH_POT_READ_PERIOD; // aggiornamento timer
317     pot_read = analogRead(RH_POT); // 1. lettura potenziometro umidità
318
319     // 2. controllo threshold
320     if (abs(status.rh_pot_value - pot_read) >= RH_POT_THLD) {
321         status.rh_pot_value = pot_read; // aggiornamento valore potenziometro umidità
322
323         // 3. calcolo nuovo setpoint di umidità
324         new_rh_setpoint = map(pot_read, RH_POT_MIN, RH_POT_MAX, RH_MIN_SP / RH_STEP_SP, RH_MAX_SP /
325                               ↪ RH_STEP_SP) * RH_STEP_SP;
326         if (new_rh_setpoint < RH_MIN_SP) new_rh_setpoint = RH_MIN_SP; // controllo limite inferiore
327         if (new_rh_setpoint > RH_MAX_SP) new_rh_setpoint = RH_MAX_SP; // controllo limite superiore
328
329         // stampa su display lcd
330         lcd_new_rh_setpoint(new_rh_setpoint);
331
332         // 4. controllo variazioni del setpoint di umidità
333         if (status.rh_setpoint != new_rh_setpoint) {
334             timers.rh_setpoint = millis(); // aggiornamento timer
335             set_auto_ctrl(); // attivazione controllo automatico
336         }
337     }
338
339 // 5. aggiornamento effettivo del setpoint di umidità
340 if (status.rh_setpoint != new_rh_setpoint && millis() - timers.rh_setpoint >= SP_UPDATE_DELAY) {
341     status.rh_setpoint = new_rh_setpoint; // aggiornamento setpoint precedente
342     serial_new_rh_setpoint(); // stampa su serial monitor
343 }
344
345
346 // -----
347 // ----- lettura dati dal sensore SHT20 -----
348 // -----
349
350 /**
351  * @brief Gestione del sensore SHT20.
352  *
353  * Come illustrato nella classe sht20, per leggere le misurazioni del
354  * sensore SHT20 è necessario invocare ripetutamente la funzione update()
355  * fino a quando non restituisce true. Tale funzione si occupa di gestire
356  * in autonomia le fasi di richiesta, attesa e lettura dei dati dal sensore.
357  *
358  * Se la funzione update() restituisce false, significa che le misurazioni
359  * non sono ancora disponibili o che si è verificato un errore. Va quindi
360  * verificata la presenza di eventuali errori con la funzione get_error().
361  *
362  * Quando la funzione update() restituisce true, significa che le nuove
363  * misurazioni sono disponibili e vanno lette entrambe tramite le funzioni
364  * get_temperature() e get_humidity(), altrimenti non è possibile effettuare
365  * la richiesta di nuove misurazioni.
366  *
367  * All'arrivo di nuove misurazioni si effettuano le seguenti operazioni:
368  * 1. lettura della temperatura e dell'umidità (in float) e conversione
369  *    dei valori in interi a 4 cifre
370  * 2. verifica di eventuali overflow dei valori e salvataggio dei valori
371  *    nella struct status
372  * 3. gestione degli attuatori se in modalità automatica
373  * 4. stampa di log sullo stato dell'incubatrice su serial monitor
374  *
375  * La gestione degli attuatori dipende dal tipo di controllo selezionato:
376  * - per i controllori ad isteresi ON/OFF, si verifica se la differenza tra
377  *   il setpoint e la misurazione supera la soglia di isteresi, modificando
378  *   di conseguenza lo stato degli attuatori
379  * - per i controllori PID, si aggiungono i nuovi dati al controllore con
380  *   la funzione pid_add_data().
381  */
382
383 // aggiorna il sensore e verifica la presenza di nuovi dati
384 if (!sensor.update()) {
385     // verifica presenza di nuovi errori nel sensore SHT20

```

```

386     if (sensor.get_error() && !err_check(ERR_SHT20)) {
387         serial_sht20_error(sensor.get_error()); // stampa errore su serial monitor
388         lcd_sht20_error(sensor.get_error()); // stampa errore su display lcd
389         err_set(ERR_SHT20); // impostazione bit di errore per il sensore SHT20
390     }
391 } else {
392     // 1. lettura della temperatura dal sensore e conversione in intero a 4 cifre
393     sensor_read = (int16_t)(sensor.get_temperature() * 100 + 0.5);
394
395     // 2. verifica overflow della temperatura e gestione errori
396     if (sensor_read < 0) { // verifica temperatura negativa
397         err_set(ERR_TEMP_OVERFLOW); // - impostazione bit di errore
398         status.temp_sht20 = 0; // - assegnazione valore minimo
399     }
400     else if (sensor_read > 9999) { // verifica temperatura troppo alta
401         err_set(ERR_TEMP_OVERFLOW); // - impostazione bit di errore
402         status.temp_sht20 = 9999; // - assegnazione valore massimo
403     }
404     else { // temperatura valida
405         err_clear(ERR_TEMP_OVERFLOW); // - rimozione bit di errore
406         status.temp_sht20 = sensor_read; // - salvataggio valore
407     }
408
409     // 1. lettura dell'umidità dal sensore e conversione in intero a 4 cifre
410     sensor_read = (int16_t)(sensor.get_humidity() * 100 + 0.5);
411
412     // 2. verifica overflow dell'umidità e gestione errori
413     if (sensor_read < 0) { // verifica umidità negativa
414         err_set(ERR_RH_OVERFLOW); // - impostazione bit di errore
415         status.rh_sht20 = 0; // - assegnazione valore minimo
416     }
417     else if (sensor_read > 9999) { // verifica umidità troppo alta
418         err_set(ERR_RH_OVERFLOW); // - impostazione bit di errore
419         status.rh_sht20 = 9999; // - assegnazione valore massimo
420     }
421     else { // umidità valida
422         err_clear(ERR_RH_OVERFLOW); // - rimozione bit di errore
423         status.rh_sht20 = sensor_read; // - salvataggio valore
424     }
425
426     // 3. controllo degli attuatori se in modalità automatica
427     if (!status.manual_ctrl) {
428         // controllo ad isteresi ON/OFF della temperatura
429         #if TEMP_CTRL == 1 // HYSTERESIS
430             if (!status.heat_relay && status.temp_setpoint - status.temp_sht20 > TEMP_HYS_THLD) {
431                 heat_turn_on();
432                 serial_auto_heat_on();
433             }
434             else if (status.heat_relay && status.temp_sht20 - status.temp_setpoint > TEMP_HYS_THLD) {
435                 heat_turn_off();
436                 serial_auto_heat_off();
437             }
438         #endif
439
440         // controllo tramite PID della temperatura
441         #if TEMP_CTRL == 2 // PID
442             pid_add_data(temp_pid, status.temp_setpoint - status.temp_sht20, status.temp_sht20);
443         #endif
444
445         // controllo ad isteresi ON/OFF dell'umidità
446         #if RH_CTRL == 1 // HYSTERESIS
447             if (!status.rh_relay && !status.refill_led && status.rh_setpoint -
448                 ↵ rh_at_temp_setpoint(status.rh_sht20, status.temp_sht20, status.temp_setpoint) > RH_HYS_THLD)
449                 ↵ {
450                 rh_turn_on();
451                 serial_auto_rh_on();
452             }
453             else if (status.rh_relay && rh_at_temp_setpoint(status.rh_sht20, status.temp_sht20,
454                 ↵ status.temp_setpoint) - status.rh_setpoint > RH_HYS_THLD) {
455                 rh_turn_off();
456                 serial_auto_rh_off();
457             }
458         #endif
459
460         // controllo tramite PID dell'umidità
461         #if RH_CTRL == 2 // PID
462             pid_add_data(rh_pid, status.rh_setpoint - rh_at_temp_setpoint(status.rh_sht20,
463                 ↵ status.temp_sht20, status.temp_setpoint), status.rh_sht20);
464         #endif

```

```

461     }
462
463     // 4. stampa di log sullo stato dell'incubatrice su serial monitor
464     serial_datatolog();
465 }
466
467
468 // -----
469 // ----- controllo PWM per attuatori binari -----
470 // -----
471
472 /**
473  * @brief Implementazione del controllo pwm per gli attuatori.
474  *
475  * Siccome gli attuatori hanno un output binario (acceso/spento), mentre
476  * un controllore PID produce un output discreto (0-100), si implementa
477  * un sistema di conversione dell'output discreto del PID in un segnale
478  * pwm binario per gli attuatori.
479  *
480  * Le variabili "temp_pwm_value" e "rh_pwm_value" contengono la larghezza
481  * del duty cycle in millisecondi per il controllo rispettivamente del
482  * riscaldatore e dell'umidificatore. È stato scelto di memorizzare il
483  * valore del duty cycle in millisecondi per evitare di dover eseguire
484  * continue conversioni da un valore discreto (0-100) a un intervallo
485  * di tempo in millisecondi.
486  *
487  * All'inizio di ogni ciclo, si effettua l'aggiornamento dei valori del duty
488  * cycle calcolando l'output del PID ed effettuando le opportune conversioni
489  * per ottenere il valore in millisecondi. Si applicano inoltre dei vincoli
490  * per evitare rapide accensioni e spegnimenti degli attuatori, impostando
491  * un periodo minimo di accensione e spegnimento oltre il quale il duty
492  * cycle viene forzato a 0 o al periodo massimo del ciclo pwm.
493  *
494  * Successivamente, si controlla lo stato degli attuatori confrontando il
495  * tempo trascorso dall'inizio del ciclo pwm con il valore del duty cycle.
496  * Se il tempo trascorso è inferiore al duty cycle, l'attuatore viene acceso,
497  * altrimenti viene spento.
498  *
499  * Il controllo pwm viene disabilitato quando è attivo il controllo manuale.
500  * Alla riattivazione del controllo automatico, il timer del ciclo PWM viene
501  * reimpostato a "millis() - PWM_PERIOD" per garantire che il ciclo pwm
502  * inizi immediatamente.
503  */
504
505 // implementazione PWM se è attivo almeno un controllore PID
506 #if TEMP_CTRL == 2 || RH_CTRL == 2
507     // controllo pwm in modalità automatica
508     if (!status.manual_ctrl) {
509         // inizio del ciclo di controllo PWM ogni CTRL_PWM_PERIOD millisecondi
510         if (millis() - timers.start_pwm >= PWM_PERIOD) {
511             timers.start_pwm += PWM_PERIOD; // aggiornamento timer
512
513             // aggiornamento dei valori del duty cycle
514             #if TEMP_CTRL == 2 // PID per la temperatura
515                 // conversione output del PID della temperatura in duty cycle
516                 status.temp_pwm_value = (pid_update_output(temp_pid) - PID_MIN_OUTPUT) * PWM_PERIOD /
517                     (PID_MAX_OUTPUT - PID_MIN_OUTPUT);
518
519                 // applicazione dei vincoli sul duty cycle
520                 if (status.temp_pwm_value < PWM_MIN_TIME_ON)
521                     status.temp_pwm_value = 0;
522                 else if (status.temp_pwm_value > PWM_PERIOD - PWM_MIN_TIME_OFF)
523                     status.temp_pwm_value = PWM_PERIOD;
524             #endif
525             #if RH_CTRL == 2 // PID per l'umidità
526                 // conversione output del PID dell'umidità in duty cycle
527                 status.rh_pwm_value = (pid_update_output(rh_pid) - PID_MIN_OUTPUT) * PWM_PERIOD /
528                     (PID_MAX_OUTPUT - PID_MIN_OUTPUT);
529
530                 // applicazione dei vincoli sul duty cycle
531                 if (status.rh_pwm_value < PWM_MIN_TIME_ON)
532                     status.rh_pwm_value = 0;
533                 else if (status.rh_pwm_value > PWM_PERIOD - PWM_MIN_TIME_OFF)
534                     status.rh_pwm_value = PWM_PERIOD;
535             #endif
536         }
537
538         // controllo del duty cycle per il riscaldatore controllato in PID
539         #if TEMP_CTRL == 2

```

```

538     if (!status.heat_relay && millis() - timers.start_pwm < status.temp_pwm_value) {
539         heat_turn_on();
540         serial_auto_heat_on();
541     }
542     else if (status.heat_relay && millis() - timers.start_pwm >= status.temp_pwm_value) {
543         heat_turn_off();
544         serial_auto_heat_off();
545     }
546     #endif
547
548     // controllo del duty cycle per l'umidificatore controllato in PID
549     #if RH_CTRL == 2
550     if (!status.rh_relay && !status.refill_led && millis() - timers.start_pwm < status.rh_pwm_value)
551         ↪ {
552         rh_turn_on();
553         serial_auto_rh_on();
554     }
555     else if (status.rh_relay && millis() - timers.start_pwm >= status.rh_pwm_value) {
556         rh_turn_off();
557         serial_auto_rh_off();
558     }
559     #endif
560 }
561 #endif
562
563 // -----
564 // ----- gestione errori e led di allarme -----
565 // -----
566
567 check_temp_range(); // controllo temperatura fuori dal range di sicurezza
568 check_rh_range(); // controllo umidità fuori dal range di sicurezza
569 update_temporized_errors(); // aggiornamento stato errori temporizzati
570 update_alarm_led(); // aggiornamento stato led di allarme
571
572
573 // -----
574 // ----- gestione refill acqua e led di refill -----
575 // -----
576
577 /**
578  * @brief Verifica necessità di refill dell'acqua.
579  *
580  * Per capire se è necesasrio un refill dell'acqua, si verifica se la somma
581  * dei seguenti valori supera REFILL_INTERVAL:
582  *
583  * 1. durata delle precedenti accensioni dell'umidificatore data dalla
584  * variabile timers.refill_counter
585  *
586  * 2. tempo trascorso dall'ultima accensione, data dalla differenza
587  * (millis() - timers.last_rh_on) moltiplicata per lo stato attuale
588  * dell'umidificatore (status.rh_relay), così da considerare solo
589  * il tempo trascorso se l'umidificatore è acceso
590  */
591
592 // verifica se è necessario un refill dell'acqua
593 if (!status.refill_led && timers.refill_counter + (millis() - timers.last_rh_on) * status.rh_relay >=
594     ↪ REFILL_INTERVAL) {
595     rh_turn_off(); // spegnimento umidificatore
596     lcd_refill_message(); // stampa messaggio di refill sul display lcd
597     serial_refill_message(); // stampa messaggio di refill su serial monitor
598     digitalWrite(REFILL_LED, LED_ON); // accensione led di refill
599     status.refill_led = true; // aggiornamento stato led di refill
600 }
601
602 // -----
603 // ----- aggiornamento display lcd con stato incubatrice -----
604 // -----
605
606 // aggiorna lo stato dell'incubatrice sul display lcd
607 lcd_print_status();
608 }
609
610 // -----
611 // ----- helper functions per il main -----
612 // -----
613
614 // attivazione del controllo manuale degli attuatori

```

```

615 void set_manual_ctrl() {
616     if (status.manual_ctrl)
617         return; // controllo manuale già attivo, nessuna azione eseguita
618
619     // modifica del flag per il controllo manuale
620     status.manual_ctrl = MANUAL_CTRL;
621
622     // imposta il riscaldatore in base allo stato dello switch
623     if (status.heat_sw && !status.heat_relay) {
624         heat_turn_on(); // accensione riscaldatore e led associato
625         serial_man_heat_on(); // stampa messaggio di accensione su serial monitor
626     } else if (!status.heat_sw && status.heat_relay) {
627         heat_turn_off(); // spegnimento riscaldatore e led associato
628         serial_man_heat_off(); // stampa messaggio di spegnimento su serial monitor
629     }
630
631     // imposta l'umidificatore in base allo stato dello switch
632     if (status.rh_sw && !status.rh_relay && !status.refill_led) {
633         rh_turn_on(); // accensione umidificatore e led associato
634         serial_man_rh_on(); // stampa messaggio di accensione su serial monitor
635     } else if (!status.rh_sw && status.rh_relay) {
636         rh_turn_off(); // spegnimento umidificatore e led associato
637         serial_man_rh_off(); // stampa messaggio di spegnimento su serial monitor
638     }
639 }
640
641 // attivazione del controllo automatico degli attuatori
642 void set_auto_ctrl() {
643     if (!status.manual_ctrl)
644         return; // controllo automatico già attivo, nessuna azione eseguita
645
646     // modifica del flag per il controllo automatico
647     status.manual_ctrl = AUTO_CTRL;
648
649     // reset vecchi dati accumulati nel PID temperatura
650     #if TEMP_CTRL == 2
651     pid_reset_accumulators(temp_pid);
652     #endif
653
654     // reset vecchi dati accumulati nel PID umidità
655     #if RH_CTRL == 2
656     pid_reset_accumulators(rh_pid);
657     #endif
658
659     // reset timer pwm
660     timers.start_pwm = millis() - PWM_PERIOD;
661 }
662
663 // verifica necessità di refill dell'acqua
664 bool check_refill() {
665     /**
666     * NOTE IMPLEMENTATIVE:
667     *
668     * Si verifica se la somma dei seguenti valori supera REFILL_INTERVAL:
669     *
670     * - durata delle precedenti accensioni dell'umidificatore data dalla
671     *   variabile timers.refill_counter
672     * - tempo trascorso dall'ultima accensione, data dalla differenza
673     *   (millis() - timers.last_rh_on) moltiplicata per lo stato attuale
674     *   dell'umidificatore (status.rh_relay), così da considerare solo
675     *   il tempo trascorso se l'umidificatore è acceso
676     */
677
678     // verifica se è necessario un refill dell'acqua
679     if (!status.refill_led && timers.refill_counter + (millis() - timers.last_rh_on) * status.rh_relay >=
680         REFILL_INTERVAL) {
681         rh_turn_off(); // spegnimento umidificatore
682         lcd_refill_message();
683         serial_refill_message();
684         digitalWrite(REFILL_LED, LED_ON); // accensione led di refill
685         status.refill_led = true; // aggiornamento stato led di refill
686     }
687
688     // restituisce lo stato del led di refill (true = refill, false = no refill)
689     return status.refill_led;
690 }

```


Bibliografia

- [1] V. Bincoletto, «Realizzazione e modellazione di un prototipo di incubatrice neonatale,»
Tesi magistrale in Bioingegneria, Università di Padova, 2025.